PS5 Dump & Image Converter
User Manual

An easy-to-follow step-by-step guide for your first safe conversions and checks – plus a complete overview of all eight tasks and every additional tool in the toolbar.

Version v1.9.97 | As of November 11, 2026 | Windows · Linux · macOS

Deutsch / English / Español / Italiano / Français

Contents

1. Welcome
2. The interface in 60 seconds
3. The safe standard workflow
4. Which task fits my source?
5. Task 1 – Convert dump folder
6. Task 2 – Convert ffpfsc
7. Task 3 – Convert exFAT
8. Task 4 – Convert ffpkg
9. Task 5 – Batch conversion
10. Task 6 – AIO conversion
11. AMPR EMU Manager (CONSOLE view)
12. Task 8 – Validator
13. Task 7 – PKG / FPKG
14. The toolbar at the top
15. Theme and background image
16. Solving common problems easily
17. Terms explained briefly
18. Quick reference for printing
19. The application on Linux
20. The application on the Mac

1 Welcome

This manual is intended for everyone who is using the PS5 Dump & Image Converter for the first time. It explains the visible buttons, the safe standard workflow, all eight tasks, and the additional tools in the top bar – in plain language. You do not need to know any technical details about file systems or container structure.

> **THE MOST IMPORTANT POINT IN ONE SENTENCE**
>
> Choose the matching task on the left, specify the source, target format, target folder, and temp folder – and only then start processing.

1.1 What you need before you start

REQUIREMENT	SIMPLE RECOMMENDATION
Windows PC	Use the program file built for Windows and start it from a normal user folder.
or a Linux system	The program also runs on Linux. How to build and start it there is described in Chapter 19. Some workflows remain reserved for Windows.
or a Mac	The program also runs on a Mac with macOS 11 or newer. How to build and start it there is described in Chapter 20. The same restrictions apply as on Linux.
Free disk space	Keep room available for the source, intermediate files, and the result. Large dumps need correspondingly more.
Working copy	For important data, always work on a copy. Keep the unchanged source stored separately.
Local temp folder	Preferably a fast local drive, for example `C:\PS5Converter_Temp`. If you choose nothing, then when a task starts on Windows the program creates the folder `Temp – Dump-Image Converter` in the root directory of the system drive (where Windows is installed, usually `C:`) and works in it.

> **SAFETY RULE FOR THE BEGINNING**
>
> Delete or overwrite the original files only after the result has been created successfully and, if needed, checked with **Task 8 – Validator**.

2 The interface in 60 seconds

On the left you choose one of the eight tasks. On the right, only the fields needed for that task appear. The large area below START shows messages and results while the work is running. Below the tasks, the sidebar shows the cover image of the selected game with its name underneath. Above the cover, the

sidebar shows the firmware the game needs and – if a backport is built in – its firmware. *Game Info* and the resources window have no system frame: you can drag their sides and lower corners to make them larger, and under Windows 11 their corners are rounded.

AREA	MEANING
SOURCE	The file or folder to be processed. You can also **drag it from File Explorer into the field** (drag & drop) – this works just as well for TARGET FOLDER and TEMP FOLDER, even when the program is running as administrator. As long as a field is empty, a gray hint shows what belongs in it (depending on the task, a dump folder or a specific file).
TARGET FORMAT	Determines whether a dump folder, `.ffpfsc`, `.ffpfs`, `.exFAT`, or `.ffpkg` is to be created. Appears only when it is needed.
COMPRESSION (PFS)	How strongly the data is packed – 1 is fastest, 9 is smallest. To get started, "6 – Balanced" is enough.
WORKERS	How many work processes run at the same time. One core always stays free; very large images are additionally throttled. Since v1.8.92 the value is set with a **rotary knob**: turn it while holding the mouse button, change it by one with the **mouse wheel** or the **arrow keys**, and reset it to the default with a **double-click**. The current number is shown in the center of the knob. Move the mouse over it and the hint shows what your setting actually does.
PRESET	Four buttons to the right of the checkboxes at the bottom of the path card set the **compression level and the core count** with one click: *Fast* (level 3, all cores), *Balanced* (level 6, half the cores), *Maximum* (level 9, half the cores), and *Low RAM* (level 3, one core – only one worker uses memory, but it takes longer). The button whose values are currently set is highlighted; if you change the level or workers by hand, the highlight goes out. The **block size stays at 65536**: smaller blocks gained 0.1% in measurement, and 64 KB is the unit with which ShadowMount+ mounts the image. In a very narrow window the buttons are dropped instead of overlapping a checkbox. The idea comes from PS5-FFPFSC-PRO 1.3.0.
VERIFY	Whether the finished image is checked after packing. "Quick" checks the structure and a sample of the blocks (about 1 %, at most 1 GiB), "Full" additionally reads everything back and takes considerably longer, "Off" checks only the structure. Task 8 can also check at any time afterward.
TARGET FOLDER	The program writes the finished result here. It is best to use a different folder than the source. The field starts empty; the selection dialog opens in the most recently used target folder.
TEMP FOLDER	Since v1.9.98 the field appears only where an intermediate folder is still created (when the conversion without an intermediate folder is switched off, or with AMPR EMU and an LZ4 asset pack, where a run may exceptionally still need an intermediate folder, Section 3.3); otherwise it stays hidden, and a folder chosen earlier remains in effect. The field starts empty; **empty means: the Windows temp folder**, without it appearing in the field. Storage for temporary working files. Best is a dedicated folder on a large drive (e.g. `D:\PS5-Temp`): in the Windows temp folder, Windows storage optimization may delete files when space is low – even a working copy in the middle of a run. If the temp folder is located there and deletion is enabled, the log warns you when a run starts.
Dump folder in the work folder	Checkbox in the path card. When repacking (for example `.ffpfsc` → `.ffpkg`), the program first unpacks the source into a temporary dump folder. By default this is placed at the *target*; if the box is checked, it moves to the work folder – then only the finished result is created on the target drive. The work folder must be able to hold the complete dump for this. Pure unpack tasks are not affected: there the dump folder *is* the result.

AREA	MEANING
Dump folder: create a working copy first	Checkbox below it. If something has to be integrated into an existing dump folder (BACKPORT, AMPR EMU, asset pack, PlayGo), the program creates a working copy without asking (for Task 1, a backup in the target folder) and works in it – the original stays unchanged, and a run does not have to wait for your "Yes". Without the check mark, you are asked as before.
Shut down computer	Checkbox in the path card. If it is checked, the computer shuts down by itself after a *successful* run – see Section 3.2.
START	Begins the selected task as soon as all required information is present.
CANCEL	Requests a controlled cancellation. Afterward, wait until the program confirms the cancellation.
Live display	At the right below the log, the load is shown once per second during a run: "CPU: 42% | RAM: 9.00 GB free (15.00 GB / 24.00 GB) | Temp: …". The **free memory** comes first, followed by used and total. If it drops below 1 GB, it reads "RAM: only … free" – at 0.8 GB, a build aborted with `MemoryError` on September 20, 2026. In that case the *Low RAM* preset or closing other programs helps.
Log area	Shows understandable status messages, warnings, and the completion of the task. Every work step appears there with "[…]"; long steps (checking, checksums, transferring, working copy) report their progress every 10 % with the remaining time and, at the end, their duration. The status line below shows the same continuously ("… – 42 % · about 3:10 left" or "… – running for 1:23" when no percentage can be measured). When you choose a source, this area also shows what is already integrated in the game (BACKPORT, AMPR EMU, asset pack, PlayGo) and whether PlayGo would be needed – nothing is switched on in the process.

Nothing is preset on opening: AMPR EMU, PlayGo, and BACKPORT are off, AMPR EMU is set to *Normal* (without pack) with the newest version without a pack, and SOURCE, TARGET FOLDER, and TEMP FOLDER are empty. If TEMP FOLDER stays empty, the first task on Windows works in the folder `Temp - Dump-Image Converter` in the root directory of the system drive (usually `C:\`); the program then creates it and enters it in the field. On Linux and macOS, the system's temp folder applies. Whatever you check or choose during a session stays in place until you quit; the next time you start, everything begins empty again. The command line (`--cli`), by contrast, continues to read the most recently saved settings.

At the top right is the toolbar with additional functions (FILEZILLA, JS LOADER, CREDITS, THEME, and more). For a normal conversion you do not need these buttons – they are explained in Chapter 14.

2.1 The second view: CONSOLE

Below the logo in the sidebar, **CONSOLE ›** switches to a second view. On the left it lists ten tools for working with the console: *1. Console & payloads*, *2. WebKit Autoloader*, *3. Library*, *4. Prospero Manager*, *5. CoolSysCent-Pro*, *6. ShadowMount+*, *7. SMPlusGui*, *8. BFpilot*, *9. AMPR EMU Manager* and *10. AMPR trace*. The cover image, "Game info" and "What else you might need" are not there. On the right is the page **Console & payloads** (see below); *2. WebKit Autoloader* and *3. Library* show their own page at this spot – none of the first three tools opens a window of its own. A second press on the same button changes nothing; *1. Console & payloads* leads back. The kernel log is available as **KLOG** at the top of the title bar. **‹ CONVERT** leads back – the selected source, the settings and a running job are kept. The program always starts in the CONVERT view – even if CONSOLE was open when it was last closed.

QR code. Below the table of *1. Console & payloads*, the *QR code* button shows, for the selected service with a web interface, a QR code for your phone containing its address (`http://<PS5-Adresse>:<Port>/`); the phone's camera app opens the page. The same button is in the header of the pages with

their own web interface. Phone and console must be on the same network; the code contains only the address, no access codes. It is generated in the program; nothing goes onto the network.

4. Prospero Manager, **5. CoolSysCent-Pro**, **6. ShadowMount+** and **7. SMPlusGui**, by contrast, do not show a page of their own: One press checks whether the payload is already running on the console, sends it itself if needed – together with the ELF loader if port 9021 is still closed – and then opens its web interface directly (Prospero Manager: port 7070, CoolSysCent-Pro: port 8086, ShadowMount+: port 10101, SMPlusGui: port 7777), exactly like *Open web interface* on "Console & payloads" (see below). Every step appears in the main log. If the payload first has to be sent, a small notice window shows what is happening – without a button, you do not have to confirm anything; it closes by itself as soon as everything is loaded. If the payload is already running, it does not appear. While one of these actions is running, the five buttons are locked: They share the ELF loader.

ShadowMount+ is a special case. Out of the box, its web interface listened only on the PS5 itself up to 1.7beta7 (`api_bind_address=127.0.0.1`); since 1.7beta8 it is `0.0.0.0` . If it listens only on the PS5, port 10101 is closed from this PC even though ShadowMount+ is running – and the button sends the payload once more anyway. If the port does not answer afterwards, the log says so and gives the reason. So first set the key `api_bind_address` in the ShadowMount+ editor (*MORE TOOLS*) to the PS5 address or `0.0.0.0` – there is no login, so not on a network you do not control.

5. CoolSysCent-Pro is the *PS5 Cooling & System Center* (fan and temperatures, system info, game library, installing packages – all in the browser). Its web interface is on port 8086 out of the box and listens on all addresses (`bind_address` `0.0.0.0`); the app sets both itself (`http_port`). If one of the two values was changed and the port stays closed as a result, the log says so and names the keys – then open the page by hand in the browser. The web interface accepts only requests that come via the IP address of the console. In the table of "Console & payloads" the app is listed like the other payloads, with its running version (the app reports it itself) and the *Open web interface* button.

8. BFpilot is a second file manager for the PS5 with a web interface on port 5905. It works like *Prospero Manager*: One press checks whether the payload (`bfpilot_v0.4.4-multilang.elf` from the folder `helloworld`) is already running on the console, sends it itself if needed – together with the ELF loader if port 9021 is still closed – and then opens its page on the right in the program (on Windows) or in the browser. According to the texts in the interface itself, it can browse files and folders, upload and download, rename, copy and delete, extract archives (`rar` , `7z` , `zip`), edit text files, install packages, search via an SSD index and load payloads through the ELF loader; its log is at `/data/BFpilot/log.txt` . The page is multilingual (German, English, Spanish, French, Italian) and follows the system language of the console. Who wrote BFpilot and under which license it is released is not stated in the file. **This path is untested on the console:** The procedure (send payload, open page) matches that of the other buttons and has only been tested against simulations. Note from the page itself: Anyone who writes directly into a folder scanned by ShadowMount+ while ShadowMount+ is running can trigger a kernel panic.

Try the transfer with a small file first: How the console's FTP server behaves with resuming, renaming and free space has not been verified in general.

Console & payloads (right-hand side)

At program start, the program looks for the PS5 itself and connects: It finds its address and saves it, queries the services, loads the ELF loader if it is missing (see below), and afterwards sends your *favorites* (see below; FTP server and kernel log out of the box), as far as they are not running. You do not have to enter a PS5 address for this. The start and the result appear in the log of the main window; if no PS5 is found, the program carries on without a console.

The first button shows the same page on the right in the view – no window of its own – and searches again with every press: first at the saved address and the FTP profiles, then with the console search

(broadcast, UDP 9302, the discovery protocol of Remote Play – the console answers even without any payload and in rest mode) and, if your router lets no broadcast through, address by address in your own network. No FTP service has to be running for this. The address found then appears in the field and is saved; only if no PS5 answers, enter it yourself and press *Check console*.

At the top, the page shows the name, state and firmware of the console – the firmware as the console itself writes it (12.00, 12.02, 12.70 …). Below that, it queries all twenty-nine known ports at the same time – ELF loader (9021 and 9020), Payload manager, FTP server, Kernel log, PKG manager, Web file manager, Garlic save manager, BFpilot, Web server, ShadowMount+, Prospero Manager, ps5upload, CheatRunner, App dumper, Game compressor, SMPlusGui, aria2, zftpd, gdbsrv, ps5debug-NG, AnyPad PS5, DPI v2 and the PS5 Cooling & System Center (port 8086), PS4Mount (port 8770), NeoEnabler (port 8765), exFAT2AMPR (port 6972), PatchDL (port 12880) and Home Hider (port 8090) – and shows the port, version and purpose for each; for the FTP server, the greeting is read along so that a hung service does not pass as "running". The version is that of the bundled file the program starts. FTP server, Payload manager, the Cooling & System Center, PS4Mount, NeoEnabler, exFAT2AMPR, PatchDL, Home Hider and ShadowMount+ report their running version themselves (ShadowMount+ only if its interface is reachable from this PC); if it differs, for example because an older payload was started from elsewhere, the bundled one is shown in parentheses after it. For ShadowMount+, the second ELF loader port (9020) and DPI v2, the version of the accompanying file is shown, even though the program does not start them there. All other services do not reveal their version. Ports and web interfaces of the eight services added on October 2, 2026 (CheatRunner through ps5debug-NG), of AnyPad PS5 and of DPI v2 come from their authors' descriptions and have not yet been measured on the console. AnyPad PS5 (bundled: 0.9.1-beta) makes Bluetooth controllers usable on the PS5; games see them as a DualSense. According to the author, confirmed on a real PS5 are DualShock 4 v2, 8BitDo SF30 Pro (Switch mode) and GameSir G7 SE; Xbox, Switch Pro and other controllers are supported but not yet confirmed. Pairing is done on its web interface, which is open to anyone on the same network without a PIN (on the console also in the *Media* row). If the console asks who is using the controller, choose, according to the author, *Quick Play/Guest* (a temporary user); a DualShock 4 has to be paired again after every power-on. Always quit AnyPad with the button on its web interface, also before you start it again. DPI v2 installs PKG files through its web interface (port 12800 according to the author, adjustable; no login, so not on a network you do not control). It is a plugin for OnionHEN, sits on the console in the folder `/data/OnionHEN/plugins` and is started by OnionHEN – *Start selected* therefore does not send it. PS4Mount (gasb07, bundled: 0.5.1) plays PS4 fake PKGs directly from a USB drive, without installing them on the console: The PKG files (game, update, add-ons) belong in the main folder of an exFAT drive; on the first run, PS4Mount places them per game under `/ps4/CUSAxxxxx` on the drive. After sending (to the ELF loader), the games appear on the home screen. Its web interface (port 8770, no login, so not on a network you do not control; on the console also in the *Media* row) shows them and removes them from the library without deleting the PKG. PS4Mount reads the kernel log: Either klogsrv (port 3232) is running, or no other payload reads `/dev/klog` – otherwise the games do not start. According to the author, it runs on any firmware with fake PKG support (tested on 13.20; below 6.00 it registers each game individually); several drives connected at the same time can slow down the console's interface. The port and web interface come from the author's description and have not yet been measured on the console. NeoEnabler (gasb07, bundled: 0.3.0) starts PS4 games on the PS5 in PS4 Pro mode, according to the author up to native 4K, but only for the games it brings patches for. It requires **ps-patch-system**: Load it first after every start of the console and then send NeoEnabler (to the ELF loader). At startup, NeoEnabler loads its game patches from the author's project on GitHub and enters them into the files of ps-patch-system; before that, it backs these up under `/data/neo-enabler/xml-backup`. On its web interface (port 8765, no login, so not on a network you do not control; on the console also in the *Media* row) you choose the patch per game. The 8 GiB of memory that some patches need only works, according to the author, on firmware 13.20 (13.60 is still untested) and with the hypervisor unlocked; the *GPU clock* tab only appears on firmware 13.20 and is off by default. The port and web interface come from the author's description and have not yet been measured on the console. According to the author, you update CheatRunner (port 9999) in this order, **before** you send the new version: First,

on the console, delete everything in `/data/cheatrunner` except the folders `cheats` and `patches` (not only afterwards). Second, delete the old CheatRunner PKG (the tile on the home screen) from the PS5. Third, clear the browser cache (*Settings → System → Browser → Clear cache*). The program deletes nothing on the console itself. What is not running, you start with *Start selected* or *Send favorites* directly from the bundled folder `helloworld`; services with their own web interface are opened by *Open web interface*. Every step appears in the log below. For ShadowMount+, the page deliberately offers no *Start selected* – it usually runs permanently from the autoload, and its web interface listened only on the PS5 itself out of the box up to 1.7beta7: There, *Open web interface* opens it from this PC only once you set the key `api_bind_address` in the ShadowMount+ editor (*MORE TOOLS*) to the PS5 address or `0.0.0.0` (since 1.7beta8 this is the default) – without authentication, so not on a network you do not control. The *6. ShadowMount+* button of the view sends the payload itself, by contrast, if port 10101 is closed (see above). If the console search does not answer but services are running, the page says so explicitly: The console is then there, but your router does not let the broadcasts through. Switching the view alone sends nothing onto the network.

Web interfaces in the program. On Windows, the web interfaces of the console – Payload manager, PKG manager, Garlic, Web file manager and the others – open inside the program and fill the whole area below the top bar; the sidebar steps aside for as long as that lasts. The header shows the name, address and state, plus *Reload*, *Open in browser* and *‹ Back* to the page you came from. If you leave the interface, it is closed. It is displayed with WebView2, the browser engine of Edge, which is part of Windows. If it is missing or the program runs on Linux or macOS, the interface opens in your browser as before; the log then names the reason.

If the ELF loader (port 9021) is closed, the page loads it by itself via the Payload manager after searching and after *Check console*: If the folder `/data/pldmgr/payloads/elfldr` is missing on the PS5, it is put there via FTP (`elfldr-ps5_v0.27.elf` together with the companion file `.json`, as the Payload manager creates it itself); if it is already there, nothing is uploaded, only loaded. If no FTP server is running, elfldr goes through the web interface of the Payload manager as before. A different file under the same name is left untouched – the log then says that elfldr was not loaded.

Favorites. The first column of the table carries a star. A click on ☆ (or the space bar on the selected row) makes the service a favorite, another click removes it again; the line above the table names who is currently a favorite. Favorites are the payloads that the program sends to the PS5 by itself – at program start and on *Send favorites*. Out of the box these are the FTP server and the kernel log; *Default* restores that. Only what is not running is sent, in the order of the table; a service that is *stuck* is not sent again (see the box below). Port 9021 is part of every send: If the ELF loader is closed, the program opens it first in the way described above; if that fails, the favorites stay unsent, and the log says so. That is why the ELF loader needs no star; ShadowMount+ and DPI v2, which the program does not start itself, carry none either. If you hover the mouse over the row, you see the short instructions. The checkbox *Send automatically at program start* (on out of the box) only switches off sending at start; *Send favorites* remains your button. The choice is saved.

After sending, the page reads the output of a service only briefly (3 seconds of silence) and then polls its port until it answers. A service runs permanently and keeps the connection to the ELF loader open; every start used to wait 30 seconds because of that.

> **“STUCK” IS SOMETHING OTHER THAN “NOT RUNNING”**
>
> For the FTP server, it is checked not only whether the port accepts connections but also whether it responds. An aborted download can put ftpsrv into a state in which it keeps accepting connections and then stays silent; reloading the payload does not help then, only a restart of the console. In this case the status light shows *stuck* and writes a warning to the log – an ordinary port scan would wrongly report “running” here.

Updates. At the top right in the header of the page, next to *Check console*, is *Updates* – it concerns the tools of the program, not the PS5, and stays usable while a service is being sent. The press opens a window and immediately asks each bundled tool's author for the latest release – the payloads in the folder `helloworld`, the WebKit Autoloader, AMPR EMU and the engines (MkPFS, UFS2Tool, LibProsperoPkg, OrbisPkgTool, PS5-PKG-Tool, PS5 Wee Tools, 7-Zip). This is the only network access; without the button, the program asks nowhere. For each tool the list shows the bundled and the latest version and a verdict: *New version* (can be ticked), *Up to date*, *Bundled is newer* (for example a pre-release build that the author has not yet published as such), *New – install by hand* (engines with their own adaptations, archives and tools without a version in the file name: only the verdict is shown here, and *Release page* opens the author's page), *Not comparable* and *Unreachable*.

Ticking is done in the first column (click or space bar); *Select new* and *Deselect* affect all at once, *Download* fetches the selection. Each file is checked against the size and SHA-256 of the release before it is put in its place (on Codeberg against the neighboring `.sha256` file); a tool made of several files (WebKit Autoloader: host as `.exe` and `.py`, installer as ELF) arrives completely or not at all. If the release has no checksum, the log says so, and only the size was checked. Files are fetched to `Werkzeug-Updates` in the program's settings folder (on Windows `%APPDATA%\PS5ImageConverterPro`), **never** into the bundled folders; nothing that is bundled is deleted or overwritten. For each pattern the program takes the highest version from both locations: The table of this page, the quick selection for ELF files and the *WebKit Autoloader* page show the new file immediately; *Start selected* and the favorites send it too. The exceptions are elfldr, ftpsrv and klogsrv: For these, the program stays with the verified bundled file in the table, when starting (including the favorites) and in its own procedures; the fetched one is available in the quick selection for sending. AMPR EMU goes, as with the existing AMPR EMU update, into the folder *AMPR EMU updates* next to the program. *Open folder* shows the storage folder; you may delete fetched files there at any time.

Without a login, GitHub allows 60 requests per hour, and a full check needs around fifty. The program remembers every response together with its ETag; a second check only asks whether anything has changed, and such requests do not count. When the limit is reached, the window names the time from which it works again and shows the last remembered state with its date for the remaining sources.

Library

The library shows your backups as *tiles* across the whole width or as a *list*. Each card carries the cover image with the platform (PS4 or PS5), the title, badges for format and integrations and four buttons; there is no column with details next to it. At the top in the header card you choose the source:

- **This PC** searches the registered *SCAN FOLDERS* including subfolders for dump folders, container files (`.ffpfsc`, `.ffpfs`, `.exfat`, `.ffpkg`) and packages (`.pkg`, `.fpkg`) – *Add folder…*, *Remove*, *Rescan*. If a registered folder is itself a dump folder, it is the one hit; its contents are not searched. A new search ends one that is still running.

- **PS5** finds the console itself and shows what is on it: the *backups* in the usual storage locations – internal, on USB drives and on the M.2 expansion – and in addition the *installed games*, PS5 titles as well as PS4 titles installed via PKG. The type entry tells them apart ("PS5 · installed", "PS4 · installed (PKG)"). System applications of the console do not appear, and a game that also exists as a backup is listed only once – as a backup. Packages (`.pkg` , `.fpkg`) are not listed by the console side.

Progress while searching. If a search takes longer than a moment – images on a cold hard disk, packages of a hundred gigabytes, the console on the network – a window with a bar and percentage opens. It names the current stage, the folder or title the program is currently reading, and – as soon as the number of finds is known – "done of total". On the PC, the stages are *Searching folders*, *Reading details*, *Loading cover images* and *Checking integrations*; on the PS5, *Looking for the PS5*, *Searching the PS5 storage locations*, *Matching title IDs* and *Reading installed titles* come before them. The percentage is a rough guide, not a promise: How long a stage takes depends on the files. *Cancel* (or the ✕) ends the search; the list stays as it was, and the status line says that it was cancelled. On the PS5, the cancellation takes effect between two FTP commands, never in the middle of a transfer. If you switch the page or restart the search, the window closes by itself; a search that finishes in a moment does not show it at all.

So that a second search is fast, the program remembers what it has read about an image or package – by path, modification time and size of the file. If the file is replaced, it reads it again; dump folders it reads fresh every time because that is cheap. In an `.exfat` image (also in an `.ffpfsc` or `.ffpfs`) it looks only into the few folders it needs – root, `sce_sys` and `fakelib` –, not into every directory of the game.

Packages. A `.pkg` or `.fpkg` carries the badge *PKG* or *FPKG* on its card. The program reads title, ID, version, firmware and cover image from the header of the package – it never loads the file, not even at a hundred gigabytes. If an entry in the package is encrypted, the corresponding details are missing; the program then fills in ID and title from the content ID and the file name. *Copy* is locked for packages: A package is installed on the console, not mounted, and the storage locations that *Copy* sends to are for images and dump folders. *Convert* works only for a `.pkg` of a PS4 title: It opens the window *PS4 PKG Dump & Image Converter* with the file already entered. The program does not convert an `.fpkg` and a PS5 package. A package carries no integrations; its card therefore shows neither badges nor a line for them.

The header card summarizes what is currently shown. Source and view are chosen with the two switches at the top right (*This PC* | *PS5* and *Tiles* | *List*); the one that currently applies is highlighted. Below it, the count line states how many games there are ("25 games · 10 PS5, 15 PS4 · 4 with BACKPORT, AMPR or PlayGo"; during a search "3 of 38 games"). Then come the search field (title, title ID, content ID or path), the filters *All*, *PS5*, *PS4* and *Backport · AMPR · PlayGo* and *Sort by*: It orders both views by title, title ID, version, format or path, descending on request. You recognize the selected card by its colored frame; a title without a cover image shows a crossed-out picture. The list also shows a small cover image for every title. The dot in front of the status line lights up in the accent color during a search, afterwards green – and in the warning color if the search on the PS5 failed.

You do not have to know the console's address for this. Like the AMPR EMU Manager (task 7), the library tries in turn the address from the settings, the saved FTP profiles, the broadcast of the console search (UDP 9302) and finally your own network. An open port is not enough – an FTP connection must be able to open a directory that exists only on a console. If the address found is not yet in the settings, the library offers to save it; if several consoles answer, it asks which one is meant. If the PS5 answers the broadcast but its FTP service does not, it says exactly that: Then the jailbreak or `ftpsrv` is missing.

The *Info & metadata* button on a card opens a window that gives the version, SDK and the **required firmware** for the entry – also for the installed games and the backups on the PS5. It sits at the button and closes with the ✕, with Escape or with a click next to it. An image cannot be read via FTP; if it is mounted, the library takes over the details that the console itself holds for this title. If the game demands more than your PS5 has, it says "BACKPORT needed" there; the library knows the console's firmware as soon as the

PS5 has been found once – that already happens at program start. For PS4 titles it names only the required *PS4 system software*: Its numbering has nothing to do with the firmware of the PS5, and BACKPORT works only on PS5 titles. As the version of a PS4 title, the higher of the two entries `APP_VER` (a patch carries its number there) and `VERSION` (a re-mastered package carries its state there) applies. If *Look up missing metadata online* is set in the settings, the library additionally shows whether a newer update exists. *Save image*, *Copy image* and *Copy details* take the cover image and the details along.

Integrations. The window also says whether a game already carries *AMPR EMU*, the *PlayGo* replacement, *BACKPORT* libraries (with their firmware folder, for example `fw10`) or an *asset pack*. On the card, the same badges appear below the title, in the list in the *Integrated* column; if this cannot be determined, a dashed *Modifications unknown* appears, and if not all badges fit in two lines, a *+N* counts how many are missing. It looks where ShadowMount+ looks, and in its order: first a folder `backports/<Title-ID>` next to the games (from 1.7beta6 a suffix may follow the title ID, for example `PPSA12345 Spielname`: within the same search path, the folder with exactly the name of the title ID wins, otherwise the first in byte order that begins with it), then the game itself – in each case first `fakelib2`, then `fakelib`. If both are in the game, the note "fakelib2 hides fakelib" appears: ShadowMount+ then reads only `fakelib2`. Into an `.ffpkg`, into an image on the PS5 and into an installed package (without a `backports` folder), the program cannot look – there it says "cannot be determined", never "none".

Backport as a separate folder. In the BACKPORT window, *Create backport folder only …* puts the backport next to the game instead of into it: You choose an output folder, and in it `backports/<Title-ID>` is created with the downgraded files (same paths as in the game), the downgraded `param.json` and the folder `fakelib`. The game itself stays unchanged. This is what ShadowMount+ requires (from 1.7beta2): Put the folder `<Title-ID>` on the console at `/data/homebrew/backports/<Title-ID>/` (or into the folder `backports` of another search path). ShadowMount+ overlays it onto the game at startup; the folder `backports` is skipped during the normal scan. A `fakelib` of the game (for example AMPR EMU) goes into the backport folder along with it, because ShadowMount+ chooses only one folder. After building, the program asks whether it should put the folder at this location on the console via FTP right away. The target firmware is preselected to match the console: the highest level that is not above the console's firmware.

Firmware on the card. At the top left above the cover image it states which firmware the game requires according to its `param.json` (*Needs FW 10.01*). If the game carries a BACKPORT, the firmware it was downgraded for follows (*Backport 4.00*), in the color of the BACKPORT badge. In the list, the same appears in the *Firmware* column. For an image, the required firmware is only available once it has been read afterwards; this happens in the background, without you having to click the entry. The library also recognizes a folder `backports/<Title-ID>` next to the games and shows it as a BACKPORT with its firmware. For an `.ffpkg`, the program reads the UFS2 image itself (without mounting, without administrator rights) and shows cover image, title, version, required firmware and the integrations; only a few hundred kilobytes are read in the process.

Starting. The highlighted *Start* button on the card starts the title on the console – via *websrv* (port 8080), which comes with the program. If a game is currently running on the PS5, it is ended in the process; the program therefore asks beforehand. If websrv does not answer, it offers to start it via the ELF loader. Installed titles and images that ShadowMount+ has mounted can be started – also from the source *This PC* if the same title is on the console. The button needs a valid title ID; if it is missing, the button is locked, and a press on it names the reason in the status line.

Besides *Info & metadata* and *Start*, every card carries *Copy* and *Convert*. What they do depends on the selected source; what does not work for an entry is locked, and a press on it says in the status line why:

BUTTON	THIS PC	PS5
Copy	*Send to PS5* (via FTP, you choose the target on the console beforehand; locked for packages). If the folder is already there, the program asks with three answers: *Yes* resumes (files that are already there in full size are skipped, started ones continue from where they stopped – only correct if the same version is there), *No* rewrites everything, *Cancel* sends nothing. If the connection breaks in the middle of a file, the program reconnects up to five times and continues writing the file where the console has it; cancelling happens only between two files.	*Download* (backups only; beforehand it states the number of files, size and the approximate duration – calculated with the measured line, around 3.4 MB/s when fetching and 2.5 MB/s when sending). An installed game is locked: It is not a file; *Start app dumper* shows the way to the PC.
Convert	*Use as source*: The entry becomes the source for the conversion. For a PS4 `.pkg`, it opens the page *Task 7 – PKG / FPKG* (PKG) with the file instead; for an `.fpkg` and a PS5 package it is locked.	Locked – copy the title to the PC first.

Below the cards is a row of buttons for what does not belong to a single title:

SOURCE	BUTTONS
This PC	*Rename all* (uniform name form; every row can be seen beforehand, a double click removes it) and *Undo* (restores the old names). You rename a single entry in the *Info & metadata* window; *Show in Explorer* is there too.
PS5	*Start app dumper* (writes an installed application to a USB drive on the console – this is also how an installed game gets to the PC: you download the folder afterwards), *Install app …* (brings a homebrew app from its release ZIP to `/data/homebrew/<ID>`, for example ProsperoEden as `PPSA99008`; see below), *PS5 file manager* (Web file manager, port 8888, for example to move from the USB drive to the internal SSD) and *Rescan*.

In the *List* view, the four buttons of the card are in a strip below the list; they apply to the selected row.

> **CANCEL TAKES EFFECT ONLY AFTER THE CURRENT FILE**
>
> This is intentional. A download cancelled in the middle of the transfer knocks `ftpsrv` on the console out: The port keeps accepting connections and no longer answers, and no reloaded payload brings it back – only a restart of the console. That is why *Cancel* remembers your wish and ends the transfer only when the current file has arrived completely.

Install app … takes the release ZIP of a homebrew app, as its author publishes it – none is bundled. The ZIP must contain exactly one app folder (for example `PPSA99008` with `eboot.bin` and `sce_sys/param.json`). The files go directly from the ZIP via FTP to `/data/homebrew/<ID>`; nothing is extracted on the PC. If the app is already there, the library asks whether it should update: Only the files from the package are replaced, everything else in the folder stays, nothing is deleted. For ProsperoEden, it also creates the empty folders `assets/keys`, `assets/firmware` and `assets/roms`. Your own files, which you have read out yourself, belong in them, as the app's instructions describe. The program does not obtain such files. The app is started on the console as soon as ShadowMount+ has read in the folder.

On the PS5, the library renames nothing, it only reads the list of installed games, and this program writes nothing into the console's system databases.

Backing up save data (Garlic Save Manager)

Backing up, decrypting and transferring between users is done by the *Garlic Save Manager* with its own interface (port 8082). There is no page of its own for it: You reach it on the page *1. Console & payloads*. There, choose the service *Garlic save manager* in the table, send it with *Start selected* (it is started in the same fast way as the other services: read along for 3 seconds, then poll the port) and call it up with *Open web interface* – on Windows on the right in the program, on Linux and macOS in the browser. Nothing is rebuilt. Important: back up before every write-back, and anyone who has created a new user on the console will find that user's save data empty; the old ones stay with the original user.

WHO THIS PROGRAM IS THANKS TO

The complete acknowledgments are behind **CREDITS**. Almost everything this program does rests on the work of others: The PFS processing is done by **MkPFS** by PSBrew and Renan Barreto, the UFS2 structure is handled by **UFS2Tool** (patched here for files over 2 GB), PS5 packages are read by **LibProsperoPkg** by SvenGDK, PS4 packages are converted by **PS4 FFPFSC**. In addition come dozens of payloads and tools from the homebrew community. The window shows the developers as cards with a GitHub profile picture, names, their works as clickable links and a thank-you – in three groups: the program, the tools and works in it, and well-known faces of the community. The pictures come with the program (folder `credits`); nothing is loaded from the network for this.

Two minutes after startup, the program shows a banner of the community **psxtools.de** at the top right below the toolbar for five minutes, and again every 30 minutes after that. A click on it opens the site in the browser, a right click only closes the banner. It does not take focus, follows the window when it is moved, and does not appear while the program is minimized.

The program itself has been under the GNU General Public License, version 3 or later (GPL-3.0-or-later, file `LICENSE`) since October 7, 2026; versions up to and including v1.9.64 were published under the MIT license. The bundled third-party components are covered by their own licenses. Their licenses are in `THIRD_PARTY_LICENSES.md` and can be opened from the CREDITS window. Anyone who passes the program on should look there – some components have their own terms.

3 The safe standard workflow

Follow the same order for every conversion. This avoids the most common selection mistakes.

1. Click the task you want on the left.
2. Choose the source with the button to the right of the SOURCE field.
3. If shown, choose the TARGET FORMAT and, if needed, the compression level and worker threads.
4. Choose a separate TARGET FOLDER. Normally you no longer need to choose a TEMP FOLDER: the conversions work without an unpacked game, and the field appears only where an intermediate folder is created (Section 3.3).
5. Check all paths once more and click START.

6. Leave the program open until the log reports a successful completion.

> **RECOMMENDED FIRST TEST**
>
> Start with a copy of a small set of test data. Check it first with Task 8. Then run a conversion and inspect the result before you use larger data.

3.1 Reading the progress correctly

While a task is running, the bar shows the overall progress, and the same value appears next to it as a percentage. For larger conversions, the status line adds phase information (such as "Phase 3/4 – …") and, where available, a line with the current CPU, RAM and temp storage usage. These displays are updated together.

Below it, the **stage line** shows each section of the run with its state and duration, for example *✓ System files 0:03 · ▶ Integration 1:30 (Measure 1:10) · ○ Write · ○ Check · total 1:33*. A check mark means done, a triangle is running right now, a circle is still waiting; after the run the line stays with the final times. The same times appear in the log as `[ZEIT]`.

> **WHILE PROCESSING**
>
> Do not force the program to close and do not disconnect the drive. If you use CANCEL, wait for the completion message. Do not keep using an incomplete result.

3.2 Shutting down the computer after the work is done

Large conversions take a long time. With the checkbox **Shut down the computer after a successful run** (in the path card) you can leave the computer alone while it works.

This is what happens when the task was **successful**:

1. A window appears and counts down 60 seconds. During that time you can object with **Cancel – keep the computer on** or the ESC key.
2. After that, the program detaches all mounted images, removes the temporary work files and writes the log to disk.
3. Only then does Windows shut down – without any further prompts.

> **NOTHING HAPPENS AFTER AN ERROR**
>
> After an error or after CANCEL, the computer stays on so that you can read the message in the log at your leisure. That is exactly why the function is tied to the *result* of the task and not merely to its end.

- The shutdown happens without prompts and also closes other programs. Unsaved work in other windows is lost – save it beforehand.

- While the checkbox is checked, **no** success message that you would have to click away appears at the end; it would wait for a confirmation that nobody gives. The success is shown in the status line and in the log.
- If the AMPR recording assistant (section 11.1) is running, the computer stays on – it still has to restore on the console what it changed. The log then says so.

The setting is saved and also applies at the next start. In command-line mode, the switch `--shutdown-on-success` takes care of this.

3.3 When there is not enough space

Before the start, the program calculates how much space the task needs – separately for the target folder and the work folder. If there is not enough, the task does **not** simply start. A window appears that states, for each folder, what is needed, what is free and what is missing.

YOUR CHOICE	WHAT HAPPENS THEN
Choose working folder…	You pick a different work folder; the calculation is then repeated.
Choose target folder…	The same for the target – the other folder can also be too tight.
Start anyway	The task starts at your own risk.

How much is needed depends on the target format: a `.ffpfsc` needs about 0.75 times the source size, `.ffpfs` and `.exFAT` 1.1 times, a `.ffpkg` 1.3 times. If the work folder and target folder are on the *same* drive, the sum is checked – viewed separately, each would look fine, and together there would still not be enough.

A game is converted **without an intermediate folder**: the program reads it directly from the source (`.exFAT`, `.ffpfs`, `.ffpfsc`, `.ffpkg` or dump folder) and writes the result in one pass. Even with **AMPR EMU**, **BACKPORT**, fake signature, DLC EMU and PlayGo, the work folder holds only the few files that the integration changes (about 5 % of the source, at least 2 GiB); the space is then needed mainly by the target. This applies equally to asset packs with zstd and with LZ4. A `.ffpkg` is created without UFS2Tool and without administrator rights.

Only where the stream does not work, a complete working copy is created in the work folder as before, and the space needed there is as large as the game itself: with a pack profile of your own `<Ordnername>_ampr_pack.toml` next to the dump folder, with a title profile with several groups (such as EA Sports UFC 6), if the source already carries a packed asset layer or contains symbolic links, and with a `.ffpkg` with names that the image cannot hold. The log states the reason ("… packing goes through an intermediate folder as before") and warns if the work folder is too small for it. The field *TEMP FOLDER* in the main view appears only where the conversion without an intermediate folder is switched off or AMPR EMU is built in with an LZ4 asset pack (there a run may exceptionally still need an intermediate folder); a folder once chosen there remains in effect, and through the “When there is not enough space” window another work folder can be chosen.

There is no window on the command line: There, the finding is reported, and `--yes` decides as it does for any other prompt.

3.4 Lots of data or texture streaming: stuttering possible

For games with a very large amount of data or texture streaming, stuttering is possible in a `.ffpfsc` or `.ffpfs`. Before either of them is created, the program therefore checks the source – regardless of its format – and asks if the game stands out:

- **Lots of data:** from 50 GiB.
- **Texture streaming:** files that typical streaming engines create (for example `.ubulk`, `.ucas`, `.utoc`, `.streamdb`, `.resS`, `.cas`), large files with “stream” or “texture” in the name, and folders such as `StreamingAssets` – these count from 20 GiB.

Folders and `.exFAT` images are scanned for this without unpacking anything. For `.ffpfsc`, `.ffpfs`, `.ffpkg` and packages, only the file size counts; for a compressed file, the real amount of data is higher, and the message then says “at least”.

YOUR CHOICE	WHAT HAPPENS THEN
Create anyway	Work continues as normal.
Choose another format	You choose a different target format that this task offers; the task starts over with the checks. If you cancel the format choice, you return to the question.
Cancel	The operation is not started.

In batch conversion, **one** question covers all affected games. The PS4 PKG window asks about the package sizes before building a `.ffpfsc`. On the command line, this question does not exist.

3.5 Before the start, after an error and when naming

Pre-check. Before the start, the program goes through the game folder once and reports *all* findings at the same time; the affected paths are in the log. For an exFAT image, names outside ASCII, symbolic links, invalid characters and names that differ only in upper and lower case are *errors* – the writer would reject them only after copying. For `.ffpkg`, `.ffpfs` and `.ffpfsc` they are a warning; there it is not proven. There are also warnings for paths deeper than 32 folder levels and for an output name over 63 bytes (for `.ffpfsc` 58) that might no longer fit after the ShadowMount+ mount path; this has not been measured on the console. A FAT32 target is also an error if a file is over 4 GiB. Batch conversion checks only the output names; the name probe reads for 20 seconds at most. A probe that itself fails never prevents the start.

Output name by template. By default the result is named like the source. Under **SETTINGS**, section *OUTPUT NAME*, you build the name from the details of the game, for example `{title} [{title_id}] v{version}` (*Astro Bot [PPSA01234] v1.0.0*). The parts are `{name}` (name of the source), `{title}`, `{title_id}`, `{version}`, `{region}`, `{fw}` (minimum firmware) and `{pack}` (zstd or lz4, empty for a game without an asset pack); each at most once, and at least `{name}`, `{title}` or `{title_id}` must appear. The example list and the preview show the name of a sample game for each format, with the bytes against the length limit; *Default* resets to `{name}`. A usable template is saved immediately; for an unusable one the reason is shown and the last saved one stays in effect. A generated name is cut to the maximum length of the format (the title first) and made suitable for Windows (no dot or space at the end; a reserved device name such as `AUX` gets an underscore). If the name matches an existing file, the program asks as usual whether to replace it; in batch conversion, a result of the same run with the same name is skipped. The template applies to every target format, `.ffpkg` included. If the details of the

game are missing, the name of the source stays. On the command line, `--ausgabename VORLAGE` takes precedence over the setting.

Leftovers after an error. If a single run has failed or was cancelled, a card shows what was left behind: the unfinished result and, next to it, `.teil`, `.neu`, `.part` and `.partial` files, with sizes. Nothing is removed unless you ask for it. The source itself, a folder around it and a folder inside it are protected twice (when offering and when removing). In batch conversion and on the command line it stays with the log. When a game folder is moved (Task 4), `sce_sys` and, at the very end, `eboot.bin` are handled, so that an aborted half folder does not look like a game.

Delete the source after success. With the checkbox *Delete the source after success* in the path card (on the command line `--quelle-loeschen`) the program deletes the source as soon as the result is finished and verified – **permanently, not to the recycle bin**. Before the start it asks once more: *Yes* deletes after success, *No* converts and leaves the source in place, *Cancel* does not start. The checkbox applies only until the next program start. The source stays in place if the result is missing or empty, if the source has changed since the start, if result and source are the same path or one lies inside the other, and if the source is a drive, a user or system folder or a link; the run still counts as successful, and the reason is in the log. In batch conversion this applies to each source individually. If the source came from an archive, the archive is deleted.

Language at startup. Without a saved choice the program follows the display language of the system; if it does not know that language, English applies. A language chosen with the language button always applies.

4 Which task fits my source?

WHAT DO YOU HAVE?	TASK	TYPICAL PURPOSE
A regular dump folder	**1**	Convert to `.ffpfsc`, `.ffpfs`, `.exFAT` or `.ffpkg`.
A `.ffpfsc` file	**2**	Unpack as a folder or convert to another supported format.
An `.exFAT` file	**3**	Unpack as a folder or convert to `.ffpfsc` / `.ffpfs` / `.ffpkg`.
A `.ffpkg` file	**4**	Output as a folder, `.ffpfsc`, `.exFAT` or again as a verified `.ffpkg`.
Several container files	**5**	Convert several inputs one after another into the same target format.
Different supported inputs	**6**	Detect the source automatically and export to the target folder.
A PS4 or PS5 package (`.pkg`)	**7**	PKG: unpack, build to `.ffpfsc` / `.exFAT`, download updates, send to the console. FPKG: unpack, convert, build a debug package.
Only check data	**8**	Verify integrity without creating a conversion result.

SET UP OR SWITCH THE AMPR EMU VERSION?

That is not one of the eight tasks on the left, but the **AMPR EMU Manager** in the **CONSOLE** view (last button) – see Chapter 11.

5 Task 1 – Convert dump folder

WHAT IS THIS GOOD FOR?

A dump folder becomes **a single file**. You need this when a game is to go on a USB stick: The console does not read a loose directory there, but a container that ShadowMount+ mounts.

The folder itself is left untouched – a copy in the chosen format is created. The only exception: With *Dump folder (with integration)* you explicitly choose "Into the original" (see below). If you want to turn an existing file back into a folder, use Task 2, 3 or 4.

Use Task 1 if your data is already available as a regular dump folder. As the result you can choose `.ffpfsc` (compressed), `.ffpfs` (uncompressed), a real `.exFAT` image or a UFS2-based `.ffpkg`.

If **AMPR EMU** (also with asset pack), **PlayGo** or **BACKPORT** is checked (Chapter 11), the list additionally contains **Dump folder (with integration)**. In that case no image is created, but a dump folder with exactly these integrations – for example, to keep using it yourself afterwards. At the start, the program asks where to integrate:

- **Into a backup (recommended)** – the dump is copied to the target folder, and only the copy is modified. Your original stays untouched; the copy needs the same amount of space once.
- **Into the original** – the integration is made directly in the folder you chose. The program explicitly warns about this: Afterwards the files are **no longer original**. BACKPORT replaces `eboot.bin` and the libraries without a backup; AMPR EMU, asset pack and PlayGo add files. This cannot be undone.

If the backup would end up where the source is (the target folder is the folder that contains your dump, or a folder inside it), the program aborts and asks for a different target folder – or for "Into the original". If a folder of the same name already exists in the target folder, you are asked before it is replaced. On the command line, the integration is always made in a backup.

1. Click **Dump folder** and choose the folder you want.
2. Open the TARGET FORMAT list and choose the format you need.
3. If needed, adjust the compression level and worker threads.
4. Choose the target and temp folders.
5. Click START and wait for the success message.

The folder must carry the game in its root: `eboot.bin` and `sce_sys/param.json` are located directly in it, not in a subfolder – this is what ShadowMount+ requires of the root directory of an image. If

`eboot.bin` is missing, the program aborts with a message before the build (for `.ffpfsc`, `.ffpfs`, `.exFAT` and `.ffpkg` alike), instead of the problem only showing up on the console.

Archive as source. In Task 1, the button **Archive…** is to the left of **Dump folder**: It chooses an archive (`.zip`, `.7z`, `.rar`), while **Dump folder** chooses the folder as usual; an archive can also be dragged onto the source field. The program unpacks it into the work folder (`ps5conv_archiv_…`, which must have at least as much free space as the size of the archive) and takes the game folder inside it – the one with `sce_sys/param.json` – as the source. Progress and result appear in the log. The program unpacks `.zip` itself; `.7z` and `.rar` are unpacked by the bundled **7-Zip** (folder `7-Zip-26.06`; if it is missing, the program looks for an installed one). The unpack folder is replaced at the next archive, and a leftover is cleaned up after twelve hours like any other work folder.

ZIP without an unpack folder. If the result is an image (`.ffpfsc`, `.ffpfs`, `.exFAT` or `.ffpkg`) and the conversion without an intermediate folder is switched on, the program no longer unpacks a `.zip`: only a small *shadow folder* with `sce_sys` and `eboot.bin` is created in the work folder – enough for cover, title, checks and naming – and the run reads the game itself directly from the archive. This saves the space and the time of unpacking; the log names the shadow folder. If the chosen route does need the game as a folder after all (target format dump folder, a route through an intermediate folder), the program unpacks the rest before the run – with a space check, and only if the archive has not changed since the shadow folder was created. `.7z` and `.rar` are always unpacked (they are usually packed solid and cannot be read at an arbitrary position), and so is an AES-encrypted ZIP.

If the archive is password-protected, the program first tries without a password, then the common passwords `pippo` and `DLPSGAME.COM` and after that your own list: under **SETTINGS**, section *ARCHIVE PASSWORDS*, you enter further passwords separated by commas; they are saved as you type. If none fits, the window asks for the password (the input field shows asterisks, and the password does not appear in the log) and asks again if one is wrong.

Task 5 (batch conversion) also accepts archives: They can be added to the sources in the file dialog (filter *Archives*) or by drag and drop and are processed in turn – each one is prepared only when it is needed (a ZIP as a shadow folder as above, otherwise unpacked) and removed again right afterwards, so there is only ever one work folder. A folder full of archives does not stand for the archives in it there; they are chosen individually. If an archive is encrypted and no password from the list fits, it counts as a failure of that source, and the others continue. The same applies on the command line: `--source Spiel.zip` for Task 1, several archives for Task 5, and `--archiv-passwort` specifies a password, which is then the only one tried.

If the memory runs out, the program restarts the run by itself – with half as many cores, up to three times – and writes this to the log. This is detected only if the engine itself reports `MemoryError`. If even one core does not help, the notice about too little memory remains.

> **IF YOU ARE UNSURE**
>
> Keep the original dump folder. `.ffpfsc` and `.ffpfs` are compact bundles; `.exFAT` and `.ffpkg` are image and container formats, respectively, for suitable further workflows.

6 Task 2 – Convert ffpfsc

WHAT IS THIS GOOD FOR?

The way back from Task 1 and at the same time the conversion between the formats. A `.ffpfsc` becomes a dump folder again – to look inside, to change individual files or as an intermediate step.

Switching to another container format also runs through here. The program does not simply repack, but unpacks and rebuilds – otherwise one more layer of nesting would result instead of a new container.

Use Task 2 if the source is a `.ffpfsc` file. From it you can create a dump folder, an `.exFAT` or an `.ffpkg` file.

1. Click **Choose file** and choose exactly one `.ffpfsc` file.
2. Choose the target format you want.
3. Set the target and temp folders.
4. Start the processing and then check the log.

EASIEST WAY TO THE CONTENTS

Choose **Dump folder** as the target format if you want to view or further edit the contained files as a normal folder structure.

7 Task 3 – Convert exFAT

WHAT IS THIS GOOD FOR?

The same for an `.exFAT` file. It is a real file system image.

You need nothing else to unpack it. The program reads the contents directly from the file with its embedded exFAT reader – without mounting, without administrator rights, on any operating system. Only if an image is built unusually and the reader fails on it does the fallback via OSFMount apply under Windows; the program says so then and can install OSFMount itself. If only the OSFMount driver is missing, item 19 under “What else you might need” sets it up without a download – with administrator rights; Windows may ask whether to install the device software from PassMark.

Under Windows, an `.exFAT` can still be mounted as a drive – handy for looking inside and for tools that expect a drive letter. For passing it on, a `.ffpfsc` is usually the better choice – it is compressed.

Use Task 3 if your source is an `.exFAT` file. The available targets are dump folder, `.ffpfsc`, `.ffpfs` and `.ffpkg`.

1. Choose the `.exFAT` file as the source.
2. Decide on the target format you want.
3. Choose the target and temp folders.
4. Click START and let the processing finish completely.

> **WITH LARGE IMAGES**
>
> Allow enough time and free storage space. An almost full drive is a common cause of aborted operations.

8 Task 4 – Convert ffpkg

> **WHAT IS THIS GOOD FOR?**
>
> The way back for `.ffpkg`. This format carries a UFS2 file system and comes from a different tool family; here it is unpacked or converted into one of the other formats.
>
> Building an `.ffpkg` again as an `.ffpkg` sounds pointless, but it is not: The path goes through a complete unpacking and rebuild and cleans up a damaged layout along the way.

Use Task 4 if an `.ffpkg` file is available. You can choose a dump folder, `.ffpfsc`, `.exFAT` or again `.ffpkg`.

1. Choose the `.ffpkg` file as the source.
2. Choose the result format you want.
3. Set the target and temp folders.
4. Start and watch for the messages about the image check.

> **SPECIAL FEATURE OF .FFPKG**
>
> A newly created `.ffpkg` is checked automatically after writing. Only when this check has completed successfully is the result considered finished.

9 Task 5 – Batch conversion

WHAT IS THIS GOOD FOR?

When **several** sources are to get the same target – dump folders as well as image files. Instead of clicking the same thing five times, you choose all sources at once, specify a target format and let it run.

Each source is processed individually and checked individually; if one fails, the others still run to the end. The progress bar shows the overall status, the log the status per source. When the next game begins, the sidebar shows its cover and name, and the log contains a line "Next game 3/7: Name · Size · dump folder → .ffpkg". If a game needs a working copy, it is removed right after that game – the space is ready for the next one.

Use Task 5 if several dump folders or supported container files (`.ffpfsc` , `.exFAT` or `.ffpkg`) are to receive the same target format. This saves repeated individual steps.

1. Click **Choose sources**. The program asks whether you want to choose **dump folders** or **image files**; the kind chosen last is preselected.
2. For **dump folders**, the Windows folder dialog opens: Select *several* folders with Ctrl or Shift and click **Select Folder**. For **image files**, select all the files you want as usual.
3. Choose a common target format.
4. Set the target and temp folders.
5. Start the batch and let the program process all sources one after another.

A FOLDER FULL OF DUMPS

You do not have to mark the dumps individually. If you choose a folder that is not itself a dump – for example *Game Dumps* –, the program takes all the dump folders and image files in it (one level deep). If you mark nothing in the dialog, the folder the dialog is currently in applies. What you have chosen is named by the status line (count) and the log (names); the display below shows the total size. Duplicates are dropped.

Under Linux and macOS, the system dialog with multiple selection does not exist: There you choose the folders one by one and are asked each time whether another is to be added.

BEFORE A LARGE BATCH

Test a single source first. Also check whether the target folder has enough space for all results; for dump folders, the program measures their size in the background as soon as you have chosen them.

10 Task 6 – AIO conversion

WHAT IS THIS GOOD FOR?

The task for all cases – and the difference from 1 to 4 is exactly one: **You do not have to know what you have in front of you.** Task 1 expects a folder, Task 2 a `.ffpfsc`, Task 3 an `.exFAT`. Task 6 takes any of these and detects the source by itself.

This changes nothing about the result: Behind it are the same paths as in the individual tasks. If you regularly do the same conversion, the matching individual task gets you to the goal a little faster, because the target format is already preselected there.

Task 6 (universal export) is handy if you want to select a supported source and export it directly to a desired format. Possible sources are dump folders as well as `.ffpfsc`, `.exFAT` and `.ffpkg`.

1. Choose the input via **Choose source**.
2. Choose dump folder, `.ffpfsc`, `.ffpfs`, `.exFAT` or `.ffpkg` as the target.
3. Set the target and temp folders.
4. Start and check the detected source in the log.

WHEN TASK 6 MAKES SENSE

Use the universal export if you want to process various supported sources with the same clear workflow. If you already know exactly which source file you have, Tasks 1 to 4 are just as suitable.

BUILD THE SAME FORMAT AGAIN – WITH INTEGRATION (SINCE V1.9.35)

You have a finished image and want to add an **AMPR EMU asset pack**, the **PlayGo stub** or **BACKPORT** to it afterwards? Choose the same file as the source and **the same format as the target too**, check what is to go in, and start.

This works in **Tasks 2 to 6** and for **every** image format. Without an integration, the target is left out of the selection list: The run would then be a copy of the same file and would only cost time. Two formats are exempt and are always selectable, because they are rebuilt completely anyway and verified in the process – you can recognize them by the addition in the list:

FORMAT	SAME SOURCE AND TARGET?
`.ffpfsc` / `.ffpfs`	with asset pack, PlayGo or BACKPORT
`.exFAT`	**always** – in the list as ".exFAT (rebuild)"
`.ffpkg`	**always** – in the list as ".ffpkg (re-validate)"

What happens in the process: The image is unpacked into a temporary folder, the integration is made there, and the new image is built from it; the folder disappears again afterwards. You therefore need the game size once in free space in the work folder. There is no prompt – wrapping an image in an image of the same format would be nonsense; the rebuild is the only sensible way.

Why not write directly into it? A finished exFAT image from this program contains *no* free space – measured on a real image: 95 allocation units, 95 of them used, zero free. Adding a file afterwards would mean enlarging the image and rewriting the boot area, file allocation table and allocation bitmap in the process – that is "rebuilding", just with more risk. In addition: The packing tool works on a real folder, not on an image.

Up to v1.9.33, for an `.exFAT` you had to start **twice** – first to a dump folder, then back. That still works if you want to edit the folder yourself in between.

11 AMPR EMU Manager

SINCE 12.10.2026 IN THE CONSOLE VIEW

The AMPR EMU Manager is no longer a numbered task in the CONVERT sidebar, but the last button of the sidebar in the **CONSOLE** view ("9. AMPR EMU Manager"). A click on it shows the page directly on the right in the CONSOLE view, together with "Console & payloads", the WebKit Autoloader and the Library – the CONVERT view is no longer displayed in the process. Everything else in this chapter (placement path, methods, workflow) applies unchanged; only the place where the page appears has changed.

WHAT IS THIS GOOD FOR?

What is an AMPR EMU? When a game starts, the PS5 asks its APR file resolver where the game files are located. If the game runs from a mounted image instead of the internal drive, it gets no usable answer – and the game does not start. The AMPR EMU is a replacement library that answers this question on its behalf.

The AMPR EMU Manager places this library in the game folder, backs up the bundled version as `.orig` and rebuilds the index through which ShadowMount+ finds the files. You need this for every game that is to run from an image – and only for that.

BUTTON 9 SHOWS A PAGE IN THE RIGHT-HAND AREA OF THE CONSOLE VIEW

Button 9 **no longer opens a separate window** (up to v1.9.65 there were two). On the right in the CONSOLE view, where "Console & payloads", the WebKit Autoloader or the Library otherwise appear, the **AMPR EMU Manager** page appears:

- at the top the **placement** (per game, global, emulators – see section 14.9), the chosen path carries a check mark;
- below it **AMPR EMU – new method** (from 1.7 alpha8) and **AMPR EMU – old method** (up to alpha6), each with a button **Guide** next to it that opens the description for exactly this version.

Once you choose a method, the same page switches to the **workflow**: scope, what is happening now and a log; **CONFIG.INI** opens the editor, **RUN AGAIN** starts the run once more, **‹ Choose method** leads back. A second press of button 9 changes nothing; if you choose another page of the CONSOLE view, this page is gone. Short prompts (Yes/No, version choice, game folder) still come as a dialog above the main window.

The AMPR EMU Manager is not a normal format conversion. It prepares the source and then opens a manager. There you can add or remove a `fakelib` folder as well as files or folders in the root directory.

1. First create a separate working copy of your source.
2. Choose the source, target folder and temp folder.
3. Click START and wait for the manager dialog.
4. There, choose exactly one action and read every confirmation prompt carefully.

DELETING IS FINAL

The manager can remove selected contents. Therefore never use this task on your only original copy. If you are unsure, cancel the dialog.

11.1 Asset packs – storing the game files packed

> **THE VOLUMES ARE CREATED WHEN *CREATING* AN IMAGE – OR AT THE PUSH OF A BUTTON**
>
> Both dropdown lists belong to the integration performed when an image is created (tasks 1 to 6). The AMPR EMU Manager does **not** build any volumes on its own: it replaces the library, backs it up, removes it, or rebuilds the index – an existing asset pack is left untouched in the process.
>
> Since v1.9.36 there is a dedicated button for this, **Build asset pack** (section 11.2). It runs only when the button is explicitly pressed, never as a side effect of another action in the Manager.
>
> A button *Remove asset pack* has been dropped and will not come back: it deleted the manifest and volumes on the assumption that the original files were lying next to them, and since the choice described below that is no longer true in every case.

Next to the dropdown list for the AMPR build, there has been a second one with two entries since v1.9.8: **Normal** and **Asset pack**. It selects how the game files are stored – the library itself goes into the folder the same way in both cases.

> **BOTH DROPDOWN LISTS MUST BE SET CORRECTLY**
>
LIST	WHAT TO SELECT
AMPR build	`0.4.2.1 test-pack` or `0.4.2.1 test-debug-pack`
Method	**Asset pack**
>
> The third build, `0.4.2.1 test-nopack`, **cannot** do it – and that is exactly the one that appears at the top of the list by default as the newest build without debug output. The selection must therefore be changed actively. Use `test-pack` for normal use; `test-debug-pack` additionally writes a log and is intended only for troubleshooting.

> **WHAT THE DIFFERENCE IS**
>
> **Normal** is the previous route: the game files stay as individual files, exactly as they are in the dump.
>
> With the **new method** they additionally go into packed volumes (`ampr_assets-*.pak`), which the AMPR EMU reads block by block from build 0.4.2.1 on. This saves space on the disk and file descriptors on the console. Nothing else needs to be done for this: writing the profile, packing, verifying, and placing the volumes in the game folder all happen in a single run.

Verification is against the source, not just against itself. Every block that is converted back is compared byte for byte with the original file. Only when this matches for every packed file is anything transferred to the game folder. A check without this comparison would only say that the volumes are consistent in themselves – not that they carry the game content.

Your original files stay in place – unless you say otherwise. At the start, the program asks once whether the packed originals should be left out of the result; the default is *No*. Removal happens only in the working copy, never in your own dump folder. On the command line the switch is `--ampr-originale-weglassen`. With AMPR EMU NG and zstd this question is skipped: the packed originals are

always left out of the working copy. The same applies to NG with LZ4 when packing without an intermediate folder (Section 3.3); with the drakmor runtime the originals stay in the image, even if you chose “leave out”.

> **CONSOLE TEST FIRST, THEN LEAVE OUT**
>
> Whether a game really reads every file through the AMPR EMU is only shown by a run on the console – some titles bypass it (for example via `mmap`), and this goes unnoticed on the PC. The developer of the pack tools therefore requires the **complete console test with the originals** before any file disappears. So build the image with the originals first, play through it, and only then build it without.
>
> This saves space only with `.ffpfs`, `.exFAT` and `.ffpkg`. An `.ffpfsc` compresses better than the volumes: with an asset pack it was never smaller in any measurement than without.
>
> **Since v1.9.31**, files that the system itself reads always stay as ordinary files: `cache_ps5/playgo.pgm`, `*.packman`, the `pgc_*_dummy_file` files and the `movies/` folder. Until then they went into the volumes as well and were deleted when the originals were left out – but PlayGo reads them bypassing the emulation, and the game crashed about one second after starting. In addition, after packing the program inspects the finished manifest and aborts if such a file is still in it.

SOME GAMES CANNOT RUN AT ALL WITHOUT THE ORIGINALS

The volumes serve only read operations that go through APR. Whatever a game engine reads with ordinary file access must be present as a real file in the image – otherwise it is simply gone after the originals are left out.

Measured on the console (September 20, 2026). A Unity game, built without the originals: the emulator reported "No such file or directory" for five files of the Unity runtime – so it did not serve them from the volumes –, the whole capture contained not a single `apr.pack` line, and the game exited shortly after starting. The same game with the same volumes *and* the originals alongside ran through.

Since v1.9.33 the program recognized such titles by their runtime files (`data.unity3d`, `globalgamemanagers`, `global-metadata.dat`, `ScriptingAssemblies.json`, `RuntimeInitializeOnLoads.json`) and left the originals in place.

That was not enough – measured on September 21, 2026. *Arkanoid – Eternal Battle* is not a Unity title; the detection found nothing, the originals were removed, and the image crashed on the console. The same source with the originals alongside runs. On closer inspection: `.gnf` texture atlases of up to 357 MB and ten FMOD `.bank` files – FMOD reads through its own file callbacks, and large atlases are mapped into memory. That makes two out of two attempts without originals that crashed, and no title has ever shown that the volumes are read.

Since v1.9.38 the originals therefore always stay in place as long as the pack profile comes from the program – even if you chose to leave them out. A list of engines cannot do the job: it would have to know every future one already, and the failure only hits you on the console. The detection remains, but it no longer blocks – it *explains*. If Unity is detected, its name along with the files found appears in the log; otherwise the general reasoning. The image does not get smaller as a result – but it starts.

Why the program does not simply exclude these files from packing. In the image that crashed, 197 of 259 files had been packed and removed – four of the five names were among them. The capture, however, contains not a single `apr.pack` line: so it is not proven that for this title *any* packed file was served at all – the four are merely the ones where it was noticed immediately. Excluding four more names would mean guessing that the remaining 193 hold up. And `data.unity3d` carries the entire game content in many titles; if it stayed loose, nothing would be left of the pack anyway.

The way forward is through captures. The pack profile of this program is generic: it cannot know what a particular title really reads through APR. Two classes are proven never to belong in a volume – the PlayGo files and the IL2CPP metadata –, and both apply to all games of their kind. Whether a title has a third class is shown only by starting it on the console.

Tested profiles are included for four groups of games (since v1.9.78). If the program detects *ASTRO BOT*, *EA Sports UFC 6*, a game on the *Unity* engine, or one on the *Insomniac* engine when packing, it applies the matching profile instead of the general one. The profiles come from the MkPFS.CSharp project by danthespal (GPL-3.0) and were verified there on a real PS5; they are located in the folder `AMPR_PackTools-4.0\profiles`. A title is recognized by its title ID in `sce_sys\param.sfo` (ASTRO BOT, UFC 6) or by its files (Unity, Insomniac). If your own profile applies (see below), none of this is touched. A bundled profile also counts as generated by the program: the originals therefore stay in place, and on the next packing run it is rewritten. Which profile was used is named in the first line of the file `ampr_pack.toml`, whose location is shown in the log ("built-in profile for …").

How strongly to pack is set by you (since v1.9.80). Under **SETTINGS**, the section *Asset pack (LZ4)* holds the LZ4 level. The default is **9**. Level 12, which was fixed until v1.9.79, saves only 0.0 to 0.6 percentage points more according to measurements on real game data, but takes 1.2 to 3.6 times as long; level 1 packs fastest and saves hardly any space. The level changes only size and packing time, not the format – the console reads every level the same way. It applies to the program's general profile and to the bundled game profiles, *not* to your own profile (see below). On the command line it is set by `--ampr-lz4-stufe` (1 to 12).

Pack method in the main window. In the integration row, below the method (second row), there is the same *LZ4 / zstd* selection; it is the same setting as in SETTINGS. It can only be selected with AMPR EMU ticked and the method *Asset pack*; with *Normal* it stays grey and nothing is packed with LZ4 or zstd. If you choose zstd, the program takes the newest regular NG version unless an NG version is already selected; with LZ4 the version stays freely selectable.

Pack level and "Full verification". To the right of the pack method is the *pack level*: it shows the levels of the chosen method (LZ4 or zstd) and is the same setting as under SETTINGS. Next to it, *Full verification* switches how thoroughly the packs are checked on their way into the image. Off (default): the structure, headers and position of every block are always checked in full, and of the content a sample (the first and last piece of large files plus random pieces, about 1 %; the seed is shown in the log). On: every byte. This applies to zstd and to LZ4 without an intermediate folder; if the program does pack LZ4 through an intermediate folder after all, the pack tool always checks everything.

LZ4 without an intermediate folder. LZ4 packs, too, are created without unpacking the game: the program compresses each block twice, once to measure and once when writing into the image, and holds each block against the measurement. The source is read twice in the process. With one track per group, the volumes are byte for byte the same as those of the pack tool. With **ampr-emu-ng (NG)** the default profile packs everything except the system files (`eboot.bin` , modules, `sce_sys` , `sce_module` , `fakelib` , `.ini` , the PlayGo files and `global-metadata.dat`); images, sound, video and configuration files therefore go into the volumes as well, because NG serves them like any other file. The packed originals are missing from the result; empty files remain as files in the image. With the drakmor runtime the more cautious default profile with the loose extensions applies (see below). If a run has to pack through an intermediate folder after all (Section 3.3), the originals stay, as with the answer No to the question. **Checked on the console** is NG with LZ4 without originals on two Unity games; the same image without the emulator shows only a black screen, so the volumes are served by NG alone. Large games and other engines have not been measured. The speed depends on the source: it is read twice, and with a slow disk (about 50 MiB/s) that takes a good six minutes for 11 GB.

Pack method LZ4 or zstd. Under **SETTINGS**, in the same section, select the *Pack method*. **LZ4** (default) builds the volumes `ampr_assets-*.pak` and runs with all builds that read packs. **zstd** builds `ampr_zstd.index` and `ampr_zstd-NNN.pak` , packs every non-empty file except the system files (`eboot.bin` , modules, `sce_sys` , `sce_module` , `fakelib` , `.ini` files) in 256 KiB blocks and usually compresses more strongly. These packs are read *only* by **ampr-emu-ng (NG)**; the runtime by drakmor does not know them. With zstd, the version list therefore shows only NG, and a run with a different build aborts without packing anything. The *zstd level* ranges from 1 to 22 (default 5, fast; 9 balanced; 19 small and slow). As with LZ4, every block is checked against the original file before anything goes into the game folder. On the command line it is set with `--ampr-pack-verfahren zstd` and `--ampr-zstd-stufe` (1 to 22). **Custom pack rules:** If a file `<Ordnername>_ampr_pack.toml` sits next to the dump folder, it also decides with zstd what stays loose (`[pack] default_action` with `compress` , `store` or `loose` , plus `[[rule]]` with `action` , `include` and `exclude` ; the last matching rule wins, patterns like `fnmatchcase`). Instead of long lists in the rule, `include_from` and `exclude_from` can point to text files with

one pattern per line (empty lines and lines starting with `#` do not count); the names are relative to the folder of the rule file, must stay inside it (no `..`, no absolute path, no link) and are appended to `include` or `exclude` respectively. Limits: 1 MiB per file, 16 MiB and 200,000 patterns in total, of which at most 10,000 with wildcards. The same rule file can also be chosen once in the **Settings** (*Pack rules for zstd*, button *Choose …*); it applies to every game for which no file sits next to the dump folder, and only with zstd. System files always stay loose; block size, `hot` and `layout` apply only to LZ4. A faulty file aborts the run before anything is written. The tool *Asset pack check / unpack* (MORE TOOLS) also opens zstd folders. **Whether ampr-emu-ng really starts a game with zstd packs on the console has not been measured** – the packed originals are missing from the result; keep the source dump (it stays unchanged).

Unsigned programs are fake-signed (since v1.9.92). If a game's `eboot.bin` and modules are plain ELF files, it never starts on the PS5 – the loader rejects them. When a task starts and the program finds such programs in a folder, it asks whether to write them as fake-signed SELF files when creating (as ps5-dump-forge does). Under **SETTINGS** in the section *Unsigned programs* this can be switched on permanently without asking; the program then also checks images, which it unpacks for that. It writes in the working copy or the unpacked folder; in the original folder a `.orig` of every file stays next to it. Already signed programs stay untouched, and the log names every changed file.

Already compressed files stay loose (since v1.9.80). The general profile does not pack images and textures (`png`, `jpg`, `dds`, `gnf` …), archives (`zip`, `7z`, `rar` …), audio or video (`ogg`, `wem`, `bnk`, `at9`, `mp4`, `bk2` …) or configuration files (`json`, `ini`, `xml`, `txt` …). LZ4 gains little there; the console also likes to read the configuration files from the disk itself. Across 37 measured dumps (1620 GB) this affects 24% of the data, mainly archives and videos. The four bundled game profiles and your own profile are not affected by this.

Empty files are not lost (since v1.9.80). The PS5 checks some empty files (such as language placeholders) outside the game; if they are missing, the game may get stuck on a black screen. When packed originals are left out, the program therefore recreates empty files after the removal.

Your own profile always takes precedence. Place a file named after the dump folder with the ending `_ampr_pack.toml` *next to* the dump folder – for example `F:\Game Dumps\Mein Spiel_ampr_pack.toml` next to `F:\Game Dumps\Mein Spiel\`. The program uses it, never rewrites it, and no longer holds you up about it: whoever packs from captures knows what the title reads through APR. The log then says “Custom pack profile adopted”.

Does this profile have to be written by hand? No. The **AMPR trace assistant** (button *10. AMPR trace* in the CONSOLE view; it shows its page on the right and begins only with **START**) leads step by step through several real game runs on your PS5 and generates exactly this file automatically – named correctly and in the right place. Beforehand it checks whether ShadowMount+ really loads the recording library, switches over temporarily if desired and puts everything back at the end – the game must not be running during this. If the title is listed in the `config.ini` under `fakelib_exclude`, it asks before recording; it does not change this entry itself. If the program is closed while it is running, it waits for the restore and then closes by itself. A detailed, illustrated guide to the whole process (trace, checking the profile, building, leaving out the originals) is opened by the button **LZ4 & Asset Pack Guide** at the top in the title bar, to the left of **USER MANUAL** – operated exactly like this manual here. The file is also in the folder `Anleitungen` next to the program: `AMPR_Asset_Packs_Mitschnitt_Assistent.html` or `.pdf`.

TWO PREREQUISITES – THE PROGRAM TELLS YOU ABOUT THEM

WHAT CAN BE MISSING	WHAT HAPPENS THEN
A pack-capable AMPR build – `test-pack` or `test-debug-pack` is required	A `test-nopack` build does not find the volumes. The program says so in the log but does not hold you up – someone who is about to switch the build anyway should not be slowed down.
The pack tool or the LZ4 module	The selection visibly falls back to *Normal* **immediately upon choosing** – not only after half an hour of work. In the finished program file, both are included.

System files never go into a volume. `eboot.bin`, `sce_sys`, the modules and everything executable stay ordinary files. The console's loader reads them before the AMPR EMU is even loaded – if they were in a volume, the game would no longer start. Since v1.9.31 this also includes `cache_ps5/playgo.pgm`, `*.packman`, the `pgc_*_dummy_file` files and the folder `movies/`: **PlayGo reads its files bypassing the emulation.** If they were in the volume and the originals were left out, the game crashed about one second after starting.

WHAT THE CONSOLE NEEDS FOR AN ASSET PACK

The program also writes these points to the log at the end of the build – there they are given exactly for the image you have just built.

WHERE	WHAT
`config.ini` of ShadowMount+ `backport_fakelib=1`	Mounts the `fakelib` folder from the image to `common/lib`. Only this way is the built-in AMPR EMU loaded at all. **1 is the default** – if it says 0 here, the volumes remain unread.
`config.ini` `global_fakelib_priority`	The default is `game`: the build from the image wins. If it says `global` *and* your own `libSceAmpr.sprx` sits in `/data/shadowmount/fakelib`, that one wins – it must then also be pack-capable, otherwise nothing finds the volumes.
`config.ini` `fakelib_exclude` `fakelib_mode` `fakelib_default_mode`	From ShadowMount+ 1.7beta4: if the game's title ID is listed under `fakelib_exclude` (equivalent to the mode `disabled` from 1.7beta5), ShadowMount+ mounts **no** `fakelib` at all for this title – not even the AMPR EMU from the image. From 1.7beta5, the same is achieved by `fakelib_mode=<ID>:disabled` and, for all games without their own entry, `fakelib_default_mode=disabled`; the last valid entry for a title wins, and `full` or `emulators` with the title ID overrides the default. Not to be confused with `global_fakelib_exclude`, which only switches off the global `fakelib`. From 1.7beta6, the ShadowMount+ web interface sets this exception per game via "Ignore global fakelib" (takes effect from the next start of the game; the mode of the game's own `fakelib` remains unchanged).
Movie and video folders (automatic since v1.9.32)	The program reads the names **from your game folder** and excludes them in exactly that spelling – the pack tool is case-sensitive. A fixed pattern `movies/**` therefore did not match `Media/StreamingAssets/Movies/`: in a real game, 17 of 17 movies ended up in the volume that way. If a video still ends up in a volume, the program says so – but it does not abort the build, because all that is proven is that all published profiles leave movies loose.
PlayGo stub in the image	If the title declares PlayGo content (language or scenario packages, recognizable by `sce_sys/playgo-scenario.json`), the stub belongs in it – otherwise the game waits for packages that never arrive. The program asks about this before the run and says so in the log.

WHEN THE RUN ABORTS INSTEAD OF FINISHING

The console runtime has hard limits: at most 2,000,000 files, 16,000,000 blocks and 1,024 volumes. A set above that is not merely slow – it is **not loaded at all**.

After packing, the program recalculates and aborts *before* anything goes into the game folder. The log then says which limit was exceeded. Remedy: choose larger blocks or leave part of the data loose.

11.2 Building an asset pack afterwards

If you have a finished backup and want to put the game files into volumes afterwards, there is no need to convert it again. The button **Build asset pack** on the Manager page does this on the spot – for all four sources that the AMPR EMU Manager accepts anyway.

SOURCE	WHAT THE PROGRAM DOES
Dump folder	packs directly in the folder
`.ffpfsc` / `.ffpfs`	unpacks, packs, repacks
`.exFAT`	unpacks, packs, rebuilds the image
`.ffpkg`	unpacks, packs, repacks

1. Choose the source, target folder and temp folder, and click START.
2. At the top of the Manager page, select the **AMPR build** to pack with.
3. Click **Build asset pack**.

THE BUILD MUST BE ABLE TO READ VOLUMES

Packing uses the build shown in the selection field – nothing is searched for automatically. Only a build of type **test-pack** or **test-debug-pack** can read volumes. If the selection is on a different one, the window says so immediately. Without this feedback, the volumes would end up in the image and the library would not touch them.

INDEX FIRST, THEN THE PACK

The pack tool reads `ampr_emu.index` to assign the filelds. If the file is missing from the game folder, the run aborts before anything is created. In that case, first click **Rebuild index only** and then **Build asset pack**. Conversely, an index rebuilt later would render the finished volumes unusable.

THE CONTENT GROWS – THE FILE DOES NOT NECESSARILY

The AMPR EMU Manager does **not** ask whether packed original files should be removed – they always stay in place. The **content** therefore grows: the result contains originals *and* volumes.

How the *file size* changes is decided by the source. Measured again on a real image: the content grew from 260,935,330 to 271,961,880 bytes (+4.2%), but the image file became *smaller*, from 364,904,448 to 275,972,096 bytes – the original image had 28.5% slack, and the built-in exFAT writer works without free allocation units. So expect more content, not reliably a larger file.

This is intentional: if the source is a *dump folder*, the AMPR EMU Manager works directly in your folder – unlike when creating, where the result is created fresh anyway. Deleted original files could not be restored there. If you want a *smaller* image, rebuild it via tasks 1 to 6 and answer the question about the originals there.

11.3 PlayGo – the question comes before the work

Next to the **AMPR EMU** checkbox is **PlayGo**. It additionally places a stub for `libScePlayGo.sprx` in the game folder, which reports all language and scenario packages as installed. It is off by default, because most games do not need it: of 31 measured dumps, only one declared PlayGo content (`sce_sys/playgo-scenario.json`).

If a title declares such content and PlayGo is not checked, the program asks at the start of the run – *before* anything is copied, unpacked or packed – whether PlayGo should be switched on. *Yes* is preselected: the checkbox gets checked, and the stub goes into the result right away. With *No*, it continues without.

For a dump folder and for `.exFAT` , `.ffpfsc` and `.ffpfs` , the program looks into the image's directory for this – in fractions of a second, without unpacking it. A `.ffpkg` cannot be looked into this way; there the question comes as soon as the files are unpacked, and still before the result is built. On the command line, no question is asked: a note appears in the log instead, and the saved setting applies.

DLC EMU. Below the **AMPR EMU** checkbox is **DLC EMU** (drakmor/dlc_emu 0.3). It places three replacement libraries – `libSceAppContent` , `libSceNpEntitlementAccess` and `libSceGameUpdate` – in the `fakelib` folder and thereby reports additional content as present. Which ones is defined in the `dlc_emu.ini` in the game folder. An existing file is left unchanged; if it is missing, the program creates it from the content IDs in `dlcmap.json` , as a fallback from the complete ones in `eboot.bin` (without the ID of the game itself) and notes in the log that the entries need to be checked. Without a matching file, the libraries report no additional content. A game original of one of these libraries is backed up once as `.orig` . If the game has additional content – a `dlcmap.json` of the game with entries, a DLC folder of the PS5 App Dumper (`…_00-…-ac`) next to the game folder, a `dlc_emu.ini` or a filled `addcont…` folder – and DLC EMU is not checked, the program asks before the work, as with PlayGo, whether it should be switched on. This applies to dump folders and images (for images, the names `dlcmap.json` , `dlc_emu.ini` and `addcont…` count); a `.ffpkg` cannot be looked into, so there only a note appears in the log. The program copies such DLC folders into the game as `addcont0` , `addcont1` … and enters them with a mount point in the `dlc_emu.ini` ; all other additional content that merely unlocks something appears in it as a pure entitlement. Like AMPR EMU and BACKPORT, the integration works on the dump folder and thus applies to all target formats; it comes before the AMPR EMU, so that its index knows the new files.

11.4 Checking and unpacking a finished asset pack

Since v1.9.80 there is a dedicated window for this: **MORE TOOLS → ASSET PACK CHECK / UNPACK**. It is the PC counterpart to *Verify LZ4 Packs* and *Uncompress LZ4* of the console app *LZ4 Compression* and uses the same tool as the build (`ampr_pack.py` , `verify` and `unpack`). Choose the **game folder itself** – the one that holds `eboot.bin` and `ampr_assets.index` . An image (`.ffpfsc` , `.exFAT`) cannot be mounted; unpack it beforehand with task 2.

- **After selecting**, the window reads only the manifest and states how many files are packed and how many are loose, how large the volumes are, and whether the originals of the packed files are still alongside.
- **CHECK** reads every block and compares it against `ampr_assets.index.crc` . This works even if the originals have long been gone. In addition, the window says whether `fakelib/libSceAmpr.sprx` is a build that can read volumes (`test-pack` or `test-debug-pack`), whether files listed as loose are missing from the folder, and whether system files (`playgo.pgm` , `*.packman` …) are in the volume. These are notices, not errors of the volumes.

- **UNPACK …** puts the packed files back as loose files. You choose: *into a new folder* – the game folder stays unchanged, the result is a loose game without volumes –, or *into the game folder itself*; in that case manifest, checksums and volumes are deleted only after successful unpacking, and the window asks once more beforehand. `ampr_emu.index` and the libraries in `fakelib` stay as they are. Beforehand, the window checks whether there is enough free space on the target drive.

CANCEL stops the run. When unpacking into a new folder, the program then cleans up; when unpacking into the game folder, the whole set remains in place, and the run can be repeated.

12 Task 8 – Validator

> **WHAT IS THIS FOR?**
>
> The task that **changes nothing**. It looks into a folder or a container file and reports what is wrong: missing required files, a damaged file system, incorrect nesting.
>
> Useful before passing a file on and after every aborted run. Important to know: **unchecked is not a verdict.** If a check could not take place – for example because administrator rights are missing – the report says so explicitly instead of reporting the file as faulty.

Task 8 checks a dump folder or a supported container file for detectable problems. It does not create a conversion result; the outcome appears in the log.

1. Select the folder or file to be checked.
2. Select a temp folder with enough free space.
3. Click START.
4. Read the completion message and note any warnings.

> **A GOOD FIRST STEP**
>
> If you do not know a source yet, start with Task 8. A successful check is a good foundation – but it does not replace your own backup copy.

12.1 What is additionally checked for .ffpfsc and .ffpfs

These containers have a two-level structure: the container on the outside, and inside it exactly one image holding the game files. If something went wrong during creation, another level may have ended up in between by mistake – from the outside the file then looks completely normal, but it is unusable on the console.

Task 8 therefore looks one level deeper and reports in the log:

LINE	MEANING
🟢 `nesting: in Ordnung`	The game files are on the innermost level – as it should be.
🔴 `nesting: falsch verschachtelt`	Another image is there instead of the files. The file must be created again.
`inner_files` / `inner_dirs`	Number of files and folders on the innermost level.
🟡 `nesting: flach aufgebaut`	The files are directly in the container; the inner image is missing. Appears, for example, when checking an intermediate result.

The check does **not** unpack the file for this: it only reads the table of contents of the inner level – for a 392 MB file, about 750 kilobytes in under a tenth of a second.

12.2 Unpack check for .ffpfsc

The payload data of an `.ffpfsc` is stored compressed, and every block carries its own checksum. Simply reading the file does not reveal a damaged block – the storage device delivers it without complaint. Task 8 therefore additionally unpacks every block (without saving anything) and reports in the log:

LINE	MEANING
🟢 `payload: in Ordnung`	All blocks can be unpacked and match their checksum.
🔴 `payload: beschädigt`	At least one block cannot be unpacked – typical of an interrupted copy or transfer. Copy the file again or create it again.
🟡 `payload: nicht prüfbar`	The file is uncompressed (`.ffpfs`) – it has no checksums that could be compared.

This takes as long as unpacking does – several minutes for a large title. The bar shows the first half for this, and the subsequent reading for the SHA-256 checksum the second; CANCEL takes effect immediately in both parts.

Limit: An `.ffpfs`, an `.exFAT` and an `.ffpkg` carry no checksums for their payload data, and neither does a block that the engine stored uncompressed. Task 8 does not notice a corrupted bit in the middle of the game data there. Structure, size and file system are still checked; anyone who wants to be sure compares the checksum (SHA-256) of the copy with that of the original.

For an `.exFAT`, Task 8 additionally checks the beginning of the image: the name “EXFAT” in the boot sector and the checksum of the boot region. If the name is missing or the checksum does not match, the file is **damaged** (no longer just a note); the same applies to an `.ffpfs` / `.ffpfsc` with an unknown header.

Before starting, the program checks the sources of Tasks 1 to 6 for a plausible signature at the start of the file: an empty, wrongly named or truncated `.exfat` or `.ffpkg` is rejected with a message instead of “successfully” producing a small, unusable result.

UNCHECKED IS NOT A VERDICT

An `.ffpkg` is checked via UFS2Tool, and that requires **administrator rights**. If they are missing, the result since v1.9.1 is **UNCHECKED** – no longer "failed". The difference matters: *failed* means something was found; *unchecked* means nothing was looked at. It says nothing about the file. Start the program as administrator and check again.

In command-line mode, Task 8 returns the exit code **4** for this – neither 0 (OK) nor 1 (flagged).

13 Task 7 – PKG / FPKG

TWO INTERFACES BEHIND ONE BUTTON

Task 7 is not a format conversion like Tasks 1 to 6. It shows its own page on the right with two buttons at the top: **PKG** and **FPKG**. A click selects the respective interface; the selected button carries a check mark, and the one chosen last stays selected – even if you click another task in between and return to "7.".

Until the evening of 10/12/2026 these were two separate windows under **MORE TOOLS** (*PS4 PKG Dump & Image Converter* and *PS5 PKG Dump & Image Converter*) – the same interfaces, only now embedded instead of in a window of their own. The Library still opens a PS4 package for conversion via the same route (*Convert*) and selects "PKG" automatically.

7.1 PKG – unpack, convert and build PS4 packages

This page does everything with PS4 games that you need for the PS5: a **dump folder** (unpacked game), an **ffpfsc** or **exFAT image** for ShadowMount+, or a single **package** made from base and update. Select the source, **SCAN**, mark the game, select the output folder and format, **CREATE**. In addition there are **ONLINE UPDATES** and **SEND TO CONSOLE** (see below).

Where to find it: Task 7 – PKG / FPKG → **PKG**. In the Library, *Convert* opens the same page with the file for a PS4 package. A second click on "PKG" changes nothing.

Game info. As with Tasks 1 to 6, the cover of the marked game (`icon0.png` from the package or from `sce_sys`) and its title appear in the sidebar, and *Game info – updates & patches* shows title, title ID, content ID, version, firmware, region, category and platform from the `param.sfo`. For PS4 titles, the window looks for updates and patches on orbispatches.com; missing details such as publisher or cover are fetched from there by the program if looking up metadata is allowed in the settings.

How it works. The packages are unpacked with **OrbisPkgTool** (MIT license, folder `OrbisPkgTool-1.0.0`); base and updates are merged into one dump folder in the order of their versions (a later update overwrites the files of the earlier one). From that, **PS4 FFPFSC** builds the image together with the `param.json` for ShadowMount+, trophy file repair and a follow-up check. The bar shows the *overall progress* across all steps, with the elapsed time and the estimated remaining time next to it. With **Cancel** you stop the run immediately, including all helper processes beneath it. The work folder is removed in the background and the page is free again right away; whatever is still left when the program exits is cleaned up by the next run.

FORMAT	RESULT
Dump folder	An ordinary game folder with `eboot.bin` and `sce_sys`, updates already applied; with the *DLC* check mark also additional content in the dumper layout (`app/` and `addcont/…`). Requires OrbisPkgTool.
ffpfsc image	Compressed image for ShadowMount+ (*Level* and *Workers* determine compression and speed).
exFAT image	Uncompressed exFAT image.
Package (base + update)	A standalone fake package with the version of the update. Requires OrbisPkgTool and an available update (if there is none, *ONLINE UPDATES* looks for one).

Accepted sources are individual `.pkg` files, a folder full of packages (including subfolders) or an already unpacked game (then only to an image). For each title the program assembles base, updates and DLC together; if only an update is present, the log states that the base game is missing. The space required is estimated and checked before starting. When building an image, the cached game is *consumed* instead of copied – on exFAT storage devices (without links) this saves half the time and half the space; your own dump folder remains untouched.

You can scan any number of sources one after another: just select another one and press **SCAN** again. Up to v1.8.92 the second pass still showed the games of the first and then blocked every further scan; since v1.8.93 the source is checked each time. Scanning the same source again is as fast as before.

In the list of games found, the **Console** column says what a title is meant for. It is recognized by the title ID:

IDENTIFIER	CONSOLE
`CUSA`, `PUSA`	PS4
`PPSA`, `PPSS`, `PPUS`, `PPJP`	PS5
anything else – such as `NPUB` (PS3)	**unclear**, nothing is guessed

This is more than information: this page builds images from **PS4** packages. If you put a PS5 game in by mistake, the row is highlighted in color and the log contains the note pointing to Tasks 1 to 6, which are responsible for PS5 titles.

TROPHIES: PS4 TITLES DO NOT REGISTER FROM AN IMAGE

A PS4 game that starts from an image reports *Trophy registration failed (0x80551618)* and does **not** register its trophies – on every start, again and again.

This is by design. The error code comes from Sony's NPDRM check chain, and that requires a *regularly installed package*. A mounted image is not one. So it is not the fault of the image, of the conversion or of ShadowMount+.

PS5 titles are not affected. On August 22, 2026, every title on the console was checked to see whether its trophies are registered: six PS5 titles ran from images and *all six* had them registered. Only the PS4 title did not. PS5 games go through a newer trophy chain, PS4 games through Sony's old one – and only the old one requires installation.

Adding files does not help. What was tried was `npbind.dat` and `param.sfo` in `/system_data/priv/appmeta/<Title-ID>/`, the same file in the subfolder `trophy2/`, and finally the complete registration entry under `/user/trophy/conf/`, rebuilt by hand. After the game started it was still there *unchanged*: the system had not even looked at it. So the block sits in the console's program code, not in missing files.

If you need the trophies, install the game via the **Package Installer**. The same title then registers on the very first start. You can check on the console: under `/user/trophy/conf/` there is a folder for every registered PS4 title.

For everything else the image route remains the more convenient one – it saves the storage space of the installation. Just without trophies.

IF A GAME GETS STUCK ON THE LOADING SCREEN

Some titles from the image stop at a “Please wait” instead of switching to offline operation. The same package installed via the **Package Installer** shows its offline dialog and carries on.

Measured and ruled out: the storage location on the storage device, missing PlayGo content, an online query (the console was taken off the network entirely – same hang) and an incomplete image (compared file by file, nothing was missing). **The cause is open.** By no means every title is affected – trying it decides.

PS4 PKG → dump folder – a PS4 package as a game folder (old extractor)

PS4 PKG → dump folder (until October 2026 “Unpack PKG”) turns a PS4 package file (`.pkg`, fake PKG: game, patch or DLC) into an ordinary game folder with `eboot.bin` and `sce_sys`. It is unpacked with the same extractor that “PKG” above brings along.

Where to find it: Task 7 – PKG / FPKG → **PKG**, format **Dump folder**. On Windows, since October 2026 the OrbisPkgTool tool does the unpacking there, with the same protection rules as described below; this section applies to the previous, standalone extractor, which on Linux and macOS opens via *More* → *Unpack with the previous extractor*. It no longer has a menu entry of its own.

Select the `.pkg` and an output folder and press **Unpack**. A subfolder with the title ID is created in the output folder, for example `CUSA00775`; for a patch `CUSA16627_patch_01.02`, for a DLC `CUSA16627_dlc_<Label>`. Bar, size line (“x of y”) and status line show the progress; **Cancel** works at any time. Afterwards, **Open output folder** opens the result.

- **Nothing is overwritten.** If the subfolder already exists, the extractor aborts with a note.

- **No half-finished state under the right name.** The extractor first writes to a folder with the extension `.partial`; only when it reports "done" is it renamed. After an error or cancellation the intermediate folder is removed – the status line shows this.
- **Space.** As soon as the extractor reports the unpacked size, the page checks the free space and stops immediately if it is not enough. Unpacked, a package is often considerably larger than the `.pkg` (measured: 124 MB → 404 MB).
- **Path length on Windows.** The extractor fails at the Windows path limit of 260 characters. The target path may therefore be at most 140 characters long; otherwise choose a shorter output folder.
- **Platforms.** The extractor exists only for Windows (x64) and for Macs with an Apple processor. On Linux and on Intel Macs the page says that none is included.

> **PS5 PACKAGES ARE UNPACKED BY A DIFFERENT PAGE**
>
> "PKG" recognizes a PS5 package by its identifier and rejects it: the extractor of this page cannot read it. PS5 packages are unpacked and converted by **FPKG**; their structure (type, content ID, entries) is also shown by *More* → *Read package header* on "PKG". The extractor also rejects retail PS4 packages without a fake signature.

Online updates, sending to the console and package tools

Until October 2026 they had a window of their own, *PS4 PKG → OTA*; its package collection with rename, move and the tabs *Info*, *Files*, *Images* and *Internals* was dropped with the merge, as were *Repack*, *Build PKG from folder*, the deep check and the management of installed packages.

ONLINE UPDATES. Asks Sony's update server whether a newer update exists for the marked official PS4 title (title ID `CUSA…`) and downloads it on request (every part with a checksum). Only the title ID is sent; homebrew and self-built titles have no update there. The downloaded update is added to the list and applied the next time you build. **PS5 titles** (`PPSA…`) the program does *not* query: Sony's address answered 404 for 16 PS5 titles checked (a PS4 title as a control returned 200), and no method for the PS5 is publicly documented. Instead of guessing, a notice appears.

SEND TO CONSOLE. The dialog knows two routes:

- **PS5 – OnionHEN package manager (DPI v2).** The service runs on the PS5 (in *Console & payloads* as *DPI v2*) and accepts `.pkg` files up to 50 GB in blocks and installs them. DPI v2 is not a standalone ELF but a **plugin for OnionHEN**: OnionHEN must be installed on the PS5 (in a version with plugin support) and starts it itself.
- **PS4 – Remote Package Installer.** The console fetches the package from the PC: the program makes it available for this one console via a small web server (only the selected package, with a random path, and only to the console's address). *Address of this PC* is set to *automatic*; Windows Firewall asks the first time – allow access on the private network.

Enter the console's address and the port (default 12800); **Detect** asks which service answers there. `.pkg` files are sent – an unpacked game or an image cannot be sent this way. Packages are transferred unchanged.

MORE. *Check packages (checksums)* checks the selected packages; *Read package header …* opens the window *Read PS4 & PS5 PKG* (also for PS5 packages); *Unpack with the previous extractor …* appears only where OrbisPkgTool is missing (Linux, macOS; see above).

LIMITS – WHAT HAS BEEN VERIFIED AND WHAT HAS NOT

- **Sending has not been verified on a console.** Both routes are tested against replicas of the services, the DPI route derived from the plugin's web interface. Whether a particular package starts on your console is shown only by trying it.
- **Package (base + update) creates fake packages** (passcode of zeros) and has been verified on self-built mini packages, not on a PS4. Treat the result as an experiment and keep the original.
- **Windows only for dump folder and package.** OrbisPkgTool exists only as a Windows program. On Linux and macOS, ffpfsc and exFAT images remain (PS4 FFPFSC reads the packages there itself), as do scanning, the update query and sending.
- **PS5 packages** are not unpacked by "PKG" – for that there is **FPKG**; their outer structure is also shown by *More → Read package header*.
- File names with umlauts become `?` in the package – that is how the tool stores them.

7.2 FPKG – Unpacking, converting and building PS5 packages

Where to find it: Task 7 – PKG / FPKG → **FPKG**. This page opens PS5 packages (`.pkg`) and turns them into a dump folder or an image; from a game folder it builds a debug package. It has been tested with **debug packages (FPKG)** – that is also what the badge next to the package says. Retail packages are bound to the console; the page shows their type, but unpacking them is not verified.

On the left there are two pages (**Convert** and **Library**) and the **Options** button for the settings.

Convert

1. Drag a package, a game folder (with `sce_sys/param.json`) or a folder containing packages and game folders onto the dashed area, or click *Choose file* or *Choose folder*. Title, title ID, size and type appear immediately.
2. Choose the **Action** (see below) and the **Target folder**.
3. For *Create / Convert*, click a **Target format**: FFPKG, FFPFSC, FFPFS or exFAT – for a game folder also **Package** (see "Building a package").
4. *Start*. Progress and messages appear in the *Job* card; on the right, *Result* shows the name of the output, the title ID, the content ID, the estimated size and the free space in the target folder.

ACTION	WHAT IT DOES
Create / Convert	For a **game folder** with the *Package* tile, it builds a debug package (below); with any other tile it hands the folder straight to Task 1. For a **package**: it unpacks the game into a work folder (`ps5conv_ps5pkg_…`) and hands it to **Task 1** of the main window with the chosen target format. The same checks and prompts apply there as for any dump folder – for example the question whether a `param.json` with findings should be repaired – and the same checkboxes (AMPR EMU, BACKPORT). The work folder is cleaned up afterwards. While this runs, no other task may be running in the main window.
Extract	Writes the dump folder `Titel [PPSA…]` directly into the target folder. If it already exists and is not empty, the program replaces it only if *Overwrite existing* is ticked.

ACTION	WHAT IT DOES
Check	Reads the package in place and lists what the tool's checklist flags (header, checksum of the wrapper, `sce_sys` entries, license files, program identifier of `eboot.bin`). A finding does not mean the package is unusable – it says what would stand out on a console (for example a missing `license.dat` in a package with `applicationDrmType: upgradable`).
Unpack to the PS5	Reads the package and sends the files straight to the console via FTP – **no unpacked folder** is created on the PC, so a 50 GB game needs no free space there. Instead of the target folder, enter the **PS5 address** (the same as in the CONSOLE view) and the **Folder on the PS5**, for example `/mnt/usb0/Spielname`; `ftpsrv` must be running on the console (port 2121). Before starting, the program states the size and the approximate duration. If the folder already exists on the PS5, it asks with three answers: *Yes* resumes (complete files are skipped, partial ones are continued – only correct if the same version is there), *No* rewrites everything, *Cancel* sends nothing. If the connection drops, the program reconnects up to five times and continues where the console has the file. Cancelling takes effect only after the current file – cancelling in the middle of a file would leave half a file behind. Empty folders of the game are created as well. For packages only; the idea comes from fpkg2ftp (kvaps), the code is newly written.

Game Details (game folders only) opens a separate window with three tabs: all signed SELF/ELF files of the game with their authority category and type, the entries from `sce_sys/activity.json` and the trophy sets found. **Show files** (packages only) opens a file browser for the inner structure of the `.pkg` (`list-inner` of the tool): folders can be expanded, and single files or whole subtrees can be extracted – without unpacking the entire package.

Building a package

Choose a **game folder** (containing `eboot.bin` and `sce_sys/param.json`), the target folder and the **Package** tile. The program builds a **debug package (FPKG)** from it and then checks it with the same checklist as the action *Check*. Which tool does the building is determined by the **builder** in the options (see below): with **LibProsperoPkg** (`ffpfsc-pkg-tool`, default) the tool chooses the name of the output itself (`<Content-ID>-A…-V….pkg`); with **ProsperoPkgTool** it is named `<Titel> [<Title-ID>].pkg`.

- **Checked beforehand** are the content ID and title ID (from `param.json`) and `param.json` itself, with the same check as in Task 1. If it has findings, the program offers the **repair** (the old file is kept next to it). This is not a minor point: a package with `applicationDrmType: upgradable` gets no `license.dat` and shows a lock on the console; with the repair to `standard`, the test package that was built passed the check with no findings (38 of 38; without the repair, 2 findings).
- **Presets** are those the tool's author verified on a console: Kraken over the inner image with the built-in encoder (stored blocks install, but do not start), fake signature of the executable files and a new AMPR index if AMPR EMU is in the game. Compression mode and level, the number of build workers, PlayGo chunks, a byte-reproducible build and a PlayGo rebuild can be changed in the *Build* tab of the options (LibProsperoPkg only).
- **Apply backport before build** (*Build* tab, LibProsperoPkg only): patches ELF/SELF files in the game folder **before** the build to the chosen target firmware – the same step as in the standalone BACKPORT tool, just without its window. In addition, a minimum firmware (`requiredSystemSoftwareVersion` in `param.json`) can be set and **Build as update PKG (PATCH)** can be ticked (`contentType` temporarily set to `PATCH`, for an update PKG; reset after the build).
- **Space:** The work folder is located in the target folder (the package is created there too) and is then removed. On drives without hard links (for example exFAT), the tool first makes a copy of the game. The program asks if less than twice the game size is free, and declines if it is less than half.

- If a package of the same name already exists in the target folder, the program replaces it only with the tick *Overwrite existing*. A failed or cancelled build leaves nothing unfinished in the target folder.

> **WHETHER THE PACKAGE STARTS ON YOUR CONSOLE HAS NOT BEEN VERIFIED HERE**
>
> The tool states the verified status as: packages of this kind start up to firmware 11.60 with kstuff-lite 1.13 or newer. On newer firmware (for example 12.00), such a package installs but does not start – the block is on the console, not in the package. This program has only measured the build and the checklist; starting it on a console remains your own attempt.

Options

The **Options** button on the left opens the settings in five tabs, applied only with *Save*:

- **Library:** library folders that are searched for packages and game folders when saving (and optionally at every start) – *Search subfolders (any depth)* goes to any depth, unlike when dropping a single folder. Individual sources can be removed from the library (only from the list; the file stays).
- **Appearance:** show covers or not, sort order of the library (title, title ID, size).
- **Output & defaults:** default target folder; a separate **temp folder** for unpacking, building and extracting in this area – empty means the program's existing work folder applies (set it only if needed, for example when the system drive has too little space for large packages); default action and target format for new jobs; *Open the output folder after a successful job*; the **default builder** (LibProsperoPkg or ProsperoPkgTool, only for "Build package"); the **debug passcode** – empty uses the tool's default (32 zeros), otherwise exactly 32 printable ASCII characters. It applies to reading, checking, unpacking and building; a package built with a different passcode cannot be opened without it.
- **Build** (only for "Build package", most fields only with LibProsperoPkg): compression mode (Kraken, zlib or none) and level, number of build workers, PlayGo chunks, a byte-reproducible build and a forced PlayGo rebuild; **Apply backport before build** with target firmware and a separate fakelib folder (empty = bundled set); a **minimum firmware version** for `param.json`; the **Build as update PKG (PATCH)** option for an update PKG.
- **Maintenance:** export or import options as a file (a preview of the changes asks for confirmation before they are applied) and reset to the defaults – the library and its folders are kept.

Library

Collects your packages and game folders (the program remembers them). + *File* and + *Folder* add items, *Refresh* re-reads everything. The search understands words (all must match), `-wort` to exclude, `titel:`, `id:` and `größe:>50GB` (also `<`, `>=`, `<=`). On the right are the cover (`icon0.png` from the package) and the *Technical overview* from `param.json` – title, content ID, versions, required system software, SDK, category, DRM type –, which *Copy all* places on the clipboard. As in Tasks 1 to 6, the cover of the selected package and its title appear in the sidebar, and *Game info – updates & patches* shows its details (title, title ID, content ID, version, size) and looks for updates and patches. Double-clicking or *Convert* takes the package over to the *Convert* page; there the log is on the right below the result.

MEASURED AND LIMITS

- **Measured** (October 9, 2026, debug packages of 528 MB and 939 MB): unpacking 17 s and 53 s respectively, with all files at the correct size (for the second package, all 118 files also with the same checksum as the full unpack); converting the smaller one to `.ffpfsc` via Task 1 in 52 s (416.8 MB). Whether the console starts the result can only be shown by trying it there.
- **.NET 10** must be installed (the program needs it anyway). The tool `ffpfsc-pkg-tool` is located in the folder `PS5-PKG-Tool`, built per platform; on Windows it is tested, on Linux and macOS only built.
- **Space:** An unpacked game is larger than the package (measured: 939 MB → 1.55 GB). *Convert* needs the space for the work folder and the image.
- Nothing is downloaded from the internet.

14 The toolbar at the top

In addition to the eight tasks, the top bar offers further, independent tools. A normal conversion does not need any of them – they are aimed at more advanced use cases, in particular the connection to an already jailbroken PS5.

BUTTON	WHAT IT DOES
DIAGNOSTICS	Creates a diagnostic report (version, system, last log lines) to share when there are problems. Credentials are redacted automatically. At the very top is a verdict on how the window is displayed: labels that are too tight, clipped controls, visibly upscaled background images (measured by the fine detail the upscaling costs, not by file size alone), display scaling, memory and response time. Since v1.9.90 there is also an *Operation check* with an *Overall* verdict: it reports errors in the log, missing programs and .NET, leftover work folders, partial files and mounted drives (OSFMount/UFS2) together with the command to unmount them, orphaned tool processes and deviations in the Python packages. Notes do not count as a defect; warnings and errors do.
KLOG	Shows the kernel log of a network-connected PS5 live – read-only, default port 3232.
PARAM/MANIFEST	Editor for `sce_sys/param.json` and `manifest.json`.
PKG MERGER	Merges split `.pkg` files back into a single file. A running set can be cancelled; the partially written file is removed in that case.
MORE TOOLS	Drop-down menu with the additional tools (among others): **SHADOWMOUNT+** (configuration editor for the ShadowMount+ payload, load and write via FTP), **AMPR Index Builder**, **ASSET PACK CHECK / UNPACK** (11.4), **SELF Inspector** (14.1), **ELF → EBOOT.BIN** (14.13), **EDIT IMAGE** (14.14), **Devices on the network** (14.15), **Rename dump** (14.2), **Install app directly** (14.4), **DOWNLOADS** (14.5), **BACKPORT** (14.6), **ps5_autoloader** (14.8) and **Send unjail** (14.11). PS4 and PS5 packages (reading, unpacking, building, downloading updates, sending to the console) have no longer been here since October 12, 2026, but under Task 7 – PKG / FPKG; the AMPR trace assistant has been the button *10. AMPR trace* in the CONSOLE view since October 14, 2026.

BUTTON	WHAT IT DOES
FILEZILLA	Starts your FileZilla installation for exchanging files with the PS5. The program looks for where FileZilla is installed by itself – see Section 14.7. Also reachable with **Ctrl+Shift+T**.
JS LOADER	Sends JS or ELF payloads to an already jailbroken PS5 (ports 50000/9021) together with a small log server. `ftpsrv-ps5` on **port 2121** is bundled and preset.
CREDITS	Shows project information and the sources and libraries used.
THEME	Opens the color scheme selection – see Chapter 15.
SETTINGS	At the very top is the display: scaling (slider) and screen resolution (drop-down list) can be set by hand and reset to automatic with a button (Section 14.3). Below that is the background image for the main area and – independently of it – one just for the sidebar. Both areas have their own drop-down list with the bundled images; your own image can be chosen at any time and takes effect immediately. Also the storage location for downloads, the LZ4 level for asset packs in the *Asset pack (LZ4)* section (Section 11.1), the template for the output name (Section 3.5) and your own password list for archives (*ARCHIVE PASSWORDS*, Chapter 5).
DE / EN / ES / IT / FR	The language button shows the active language; each click switches the entire interface to the next one (German, English, Spanish, Italian, French) – main window, all dialogs and all log messages. This manual is available in all five languages; the **USER MANUAL** button opens it in the language of the interface.
FAQ	To the left of USER MANUAL: Opens the frequently asked questions with short answers in your browser – in the language of the interface (German, English, Spanish, Italian or French). They are included with the program; no internet connection is needed. As a PDF they are located next to the program file.
USER MANUAL	Opens this manual in your browser. It is included with the program; no internet connection is needed.
LZ4 & Asset Pack Guide	Opens, just like the button next to it, the detailed guide to AMPR asset packs and the trace assistant (see Section 11.1) – also included in the program, no internet connection needed.

SEVERAL TOOL WINDOWS AT THE SAME TIME

The windows of this bar can be kept open side by side. They stay **in front of the main window** – even when you click the main window to press the next button. Each still has its own entry in the taskbar and can be reached with **Alt+Tab**.

Because they belong to the main window, however, they move with it to the taskbar when you minimize it, and come back when it is restored.

PREREQUISITE FOR ALL NETWORK TOOLS

KLOG, SHADOWMOUNT+, FILEZILLA and JS LOADER require that your console has already been prepared using a separate jailbreak or homebrew method. This program does not perform that step itself.

WHICH FTP PAYLOAD?

An FTP payload must be running on the console. **ftpsrv-ps5 on port 2121** is bundled and preset; the program has been tried and tested with it. After each upload, the program checks whether the files are executable and otherwise warns in the log.

14.1 SELF Inspector

Shows the inner structure of an executable PS5 file – typically the `eboot.bin` of a dump, but also `.self`, `.sprx`, `.prx` or `.elf`.

Where to find it: MORE TOOLS → SELF INSPECTOR. Then choose the file.

SECTION	CONTENTS
Container	Kind (signed wrapper or plain ELF), detected magic, version and mode, key type, header and meta size, segment count.
Embedded ELF	32/64-bit, type (e.g. `ET_SCE_DYNEXEC`), machine, entry point, number of program headers.
Extended info	Authority ID with category (*Fake/Debug*, *Genuine*, *Privileged system*), program type, app version, firmware, digest.
Segments	Table with ID, offset, size in file and in memory, and the properties (signed, encrypted, compressed, blocked, ordered).

With **Copy report**, the complete analysis ends up on the clipboard as text – handy for bug reports.

Two build forms occur in real dumps. Most `eboot.bin` files are a signed SELF wrapper with an embedded ELF; there are two equivalent identifiers for this (`0x1D3D154F` and `0xEEF51454`), and both are recognized. Some dumpers store the file instead as a plain, unsigned ELF – then the segment table and extended info are absent, but the ELF header is still evaluated.

DELIBERATE LIMIT

Only unencrypted structure fields are read. Segment data is **not** decrypted and signatures are **not** verified – that would require console keys, which this project does not obtain. The file is only read, never modified; for large files, only the header area is read.

14.2 Rename dump

Suggests a descriptive name for a dump folder based on its own metadata.

Where to find it: MORE TOOLS → RENAME DUMP. Then choose the folder.

The window shows the title ID read from `sce_sys/param.json`, the title and the version, plus an assessment: ● all three details are present, ● something is missing, ● no valid title ID found. Three suggestions are offered:

SUGGESTION	EXAMPLE
Title ID only	`PPSA18089`
Title ID + title	`PPSA18089 Matchbox™ Driving Adventures`
Title ID + title + version	`PPSA18089 Matchbox™ Driving Adventures (01.000.001)`

The chosen suggestion goes into the input field below it and can be freely edited there; characters that are not allowed on Windows are removed automatically. **Rename** renames only the folder – the contents are left untouched. If a folder of that name already exists next to it, the program aborts instead of overwriting anything.

14.3 Building a PKG – the old way has been removed

Until early October 2026, the program built debug packages (`.pkg`) from a dump folder or an image under MORE TOOLS → BUILD PKG. This function has been **removed**: The packages did install on the console, but at launch the console reported CE-100096-6 – it could not derive the keys. With it went the second build path via the Sony SDK and the pre-check of a dump folder.

New since v1.9.71: Debug packages are built by **FPKG** (Task 7) – with a different tool (`ffpfsc-pkg-tool`), whose author has verified launching on consoles up to firmware 11.60. The old way is not coming back.

For game backups, the route via `/data/homebrew` and ShadowMount+ is the right one; for your own homebrew, **Install app directly** (14.4). The key data of an existing PS4 package is shown and unpacked by **PKG** (Task 7); that of a PS5 package by *More → Read package header* on the same page.

Using your own library – the `libs` folder

Next to the program there is a folder `libs` . If you have a **self-built** `LibProsperoPkg.dll` , put it in there – nothing more to do. Since the removal of "Build PKG", it only takes effect when **reading** a package (*More → Read package header* on **PKG**, Task 7). If you remove the file again, everything is as before. A file of 0 bytes is ignored.

Nothing is **overwritten** in the process: The program mirrors the tool folder into the settings folder and places your file there on top. Whether a custom library is currently in effect – and which one – is shown in the diagnostic report under *Custom libraries*, together with size and checksum. A newer version is recognized by the checksum, not by the date. A subfolder `libs/sdk` (formerly for the SDK build path) is no longer used; the report lists it as unused. The guide is located next to it as `libs/README.md` .

14.4 Install app directly

Creates a tile directly on the PS5 – without the detour via a package file.

Where to find it: MORE TOOLS → INSTALL APP DIRECTLY.

The console rejects self-built packages with `CE-100096-6` . This tool gets around that: It transfers the files via FTP and registers the tile with a small payload. There are two build forms to choose from – a tile that opens an address when tapped, and the route with its own program.

Tile under Media or Games. Where the tile appears on the home screen, you choose in the window under *Tile under*: **Media** (preset – the usual tiles such as Payload Manager or WebKit Autoload are located

there) or **Games**. The console reads this at login from `param.json` (`applicationCategoryType` : Media 65536, Games 0); it can only be changed by installing again. Tried and tested are the tile for a web interface under Media and the application with its own program under Games; the two other combinations have not yet been tried on the console – if a tile does not appear or does not start, choose the tried and tested side.

Prerequisite is that `ftpsrv` (port 2121) and the payload loader (port 9021) are running on the console. If 9021 is closed, the program automatically takes the detour via the Payload Manager. This route reports nothing back; the program then fetches the payload's log via FTP from `/data/appinst.log` and waits up to 20 seconds for it. In the log field you see this step as a line of its own.

> **IMPORTANT**
>
> The tile is created without a signature check. It survives a restart of the console only as long as the jailbreak environment is active again – please check yourself before each use.

14.5 DOWNLOADS – Getting updates and patches

Downloads game updates and older patches, sorts them into separate folders, and shows at all times what is running and what is already there.

Where to find it: MORE TOOLS → DOWNLOADS. The **Download** button in the *Game info – updates & patches* window also leads here.

Setting the storage location

On the first download, the program asks which drive the packages should be stored on. Two folders are then created there automatically:

```
<chosen location>\
├── PS5 Spiele Updates\    ← the newest version in each case
└── Patches\               ← older versions
```

The location can be changed at any time – in the download window via **Change …** at the top, or in the **Settings** under *Download storage location*.

The procedure

One step cannot be automated: the actual download address only comes into existence when **DETAILS** is clicked on the patch page, and that click is protected there by a security check. This program does not circumvent such protection measures. The procedure is therefore:

1. In the **Game info** window, click **Download**. The patch page opens in the browser, and the download window comes to the front.
2. On the page, confirm the security check and click **DETAILS**.
3. **Right-click Download Piece PKG** and choose *Copy link address*.
4. Back in the download window, click **From clipboard**.

After that, everything runs on its own: the address is checked, the file is classified, the target folder is created, and the download is started. Several addresses may be pasted at once – via **Paste address(es)**, one below the other in a text box.

Sony's official list – without the detour (since v1.9.81)

If the `sce_sys\param.json` of the selected PS5 game names the address of the update list (`versionFileUri`), *Game info* additionally shows **PS5 · Sony** rows: Sony's own versions with size and required firmware, the newest one marked with a star. This address contains an identifier for each title and cannot be calculated from the Title ID – it is therefore found only in the game itself. Like the others, the query runs only when online lookup is permitted.

Clicking **Download** in such a row opens a separate window and downloads the package *directly* – without a browser, without a security check, without pasting an address. The program fetches the directory of the parts, downloads them (three at a time) into a `.teil` file, and checks **every part against the SHA-256 from Sony**; if one does not match, it is downloaded once more, after which the run is aborted. Only when everything matches does the file receive its name (`<Content-ID>_v<Fassung>.pkg`) in the *PS5 Spiele Updates* folder (newest) or *Patches*. **Cancel** is harmless: the progress is kept, and the next click resumes. Beforehand, the program checks whether enough free space is available on the drive.

Communication is only with Sony's delivery network (`sgst` / `gst` / `gs2.ww.prod.dl.playstation.net`), using a private root certificate from Sony that the program brings along. The download goes to the computer, **not** to the console, and nothing is installed; whether the assembled file can be installed on your console has not been verified.

> **IS NANODNS RUNNING ON THE CONSOLE?**
>
> If so, add `sgst.prod.dl.playstation.net` and `gs2.ww.prod.dl.playstation.net` to the `[exceptions]` section of `nanodns.ini` (by default only `gst.prod.dl.playstation.net` is listed there); otherwise nanoDNS blocks these hosts for everything on the console that fetches updates, such as PatchDL. For this program on the PC, it makes no difference.

Update or patch?

The classification follows the marking in the Game info window: the version marked there with a star as the newest counts as an **Update**, every older one as a **Patch**. If this cannot be determined, the file is treated as an update. If a file ends up in the wrong folder afterward, the **Move** button moves it with one click.

Pausing and resuming

Cancel stops the selected download. **Retry** resumes it exactly where it stopped, instead of starting over – this also applies after a program restart.

The download goes into a file with the extension `.teil`. It only receives its proper name once the size matches. A partially downloaded file can therefore never be mistaken for a finished one by accident.

Delete partial file. If you no longer want a started download, select it and press **Delete partial file**: after a confirmation the partial file (with its resume record) is removed from the disk and the row from the list. Finished files and running downloads are left untouched. The window also lists started downloads of earlier runs as *incomplete*; pasting the same address again resumes them. The same is possible in the loading window of the update packages (after a cancel or error) and in *Game Info* with **Delete started downloads** for the game shown.

What is already there

Scan existing searches both folders and lists all packages already stored there, with size and Title ID. Useful after a restart or when the drive has been reconnected. **Open folder** shows the storage location in Explorer.

STATUS	MEANING
queued	In the list; the download starts shortly.
downloading …	Running; the *Progress* column shows the percentage.
resuming at …	A partially downloaded file was found; the download continues from there.
done	Completely downloaded and renamed.
present	Was already on the disk, nothing to do.
cancelled	Stopped; **Retry** resumes it.
error: …	The reason follows, for example no connection.
incomplete (…)	Leftover of an earlier run; pasting the address again resumes it, **Delete partial file** removes it.

14.6 BACKPORT – Lowering games to an older firmware

A PS5 game remembers which development kit (SDK) it was built with. At launch, the console compares this with its own firmware and does not start anything that is too new. If a game requires firmware 9.00 while your console is on 4.50, simply nothing happens when you tap it. **BACKPORT** lowers this value.

Where to find it: MORE TOOLS → BACKPORT. Then select the game folder.

The procedure

1. **Choose the folder.** The game's dump folder, that is, the one containing `eboot.bin` and `sce_sys`.
2. **Set the target firmware.** The choices are **4.00, 5.00, 6.00 and 7.00** – choose your console's version or a lower one.
3. **See what would happen.** On opening, the list already shows every executable file with its current SDK and what would happen to it. The **Analyse only** button repeats this without changing anything.
4. **Start the backport.** After a confirmation prompt, the program works through the list.

The settings

CHECKBOX	MEANING
Create backup (recommended)	Before the first access, places a complete, timestamped copy of the folder next to it. On by default – please leave it on.
Only needed libraries (experimental)	From the replacement set, copies only the libraries the game actually uses according to its import table, instead of always the whole set. Off by default; without a readable import table, the full set is used. The choice also applies to integration during creation.
Neutralize missing functions (stubs, experimental)	If the target firmware lacks a function that the game requires and that no replacement library can supply, the game aborts on the first call. With this box checked, BACKPORT replaces the call in the game itself with an empty return value: the game keeps running – **whether it then runs correctly is an open question**. No guarantee. Requires the export table of the target firmware (included for 4 to 7). Off by default; the choice also applies to integration during creation and to *Create backport folder only*. The log lists every changed location.
Apply the game's correction profile (if available)	Besides missing functions, some games need further byte corrections to the executable. A profile applies to **exactly one version** of a title: it is only applied if the SHA-256 sum of the unmodified original file matches and the expected bytes are found at every location – otherwise everything stays unchanged, and a second run recognizes "already adjusted". The profiles are stored as JSON per Title ID in the folder `Backport_Fakelibs/Korrekturprofile`; one for Anno 117 (firmware 4.xx) is included. No guarantee. Off by default; like the other boxes, it also applies to integration during creation.

Nothing else needs to be set; the rest runs on its own: the replacement libraries matching the target firmware are placed in a `fakelib` folder next to the game (without them, the game expects functions that do not exist on the older firmware), the libc adaptation follows the target firmware, and the coverage check appears in the log afterward.

What the list shows

`eboot.bin` and all `.prx` and `.sprx` files are processed. The `fakelib` folder is left out – the libraries in it already fit.

STATUS	MEANING
will be lowered	The SDK is higher than the target; the file will be processed.
already low enough	Nothing to do; the file stays unchanged.
no SDK version	The file has no module header; it stays unchanged.
not executable	Neither ELF nor SELF; it is skipped.
done – 9.00 → 4.00	Successfully lowered and re-signed.
error: …	The reason follows; the file stays unchanged.

Checking coverage

The coverage check always runs along. After the replacement libraries have been copied, the program reads which functions the game requires from them and compares this against what they actually offer.

The result appears in the log:

LINE	MEANING
libSceAgc: all 42 required functions are present.	This library covers what the game needs.
libSceAgc: 42 required, 3 missing.	The missing identifiers are listed below it. The game may crash at this point.
Provided by the console: libkernel, …	Libraries for which no replacement file is included. Not a finding – the firmware itself supplies them.
n without an import table	Statically linked files. They cannot be checked and are therefore named explicitly.

The check changes nothing and stops nothing; it is a finding to read afterward. Full coverage is no guarantee that the game will start – but a reported gap is a solid reason why it might not.

What is protected in the process

Work takes place exclusively in memory. A file is only replaced once unpacking, lowering **and** re-signing have all succeeded – and then in a single step. If any step fails, the original stays exactly as it was. An SDK value is never raised, only lowered.

> **IMPORTANT**
>
> The result is re-signed, but not *genuinely* signed – it only runs on a console that is already jailbroken. And whether a particular game actually starts after the backport cannot be predicted: some games require functions that simply do not exist on the older firmware. Therefore always work with the backup switched on, and keep the backup folder until you have successfully started the game on the console.

14.7 How FileZilla is found

The **FILEZILLA** button launches your own FileZilla installation. Because everyone installs it in a different place and under different folder names, the program searches in this order:

STEP	WHERE IT LOOKS
1	The most recently used path from the settings.
2	The Windows registry (the FileZilla installation entry).
3	The usual program paths, for example `C:\Program Files\FileZilla FTP Client`.
4	**Every folder whose name contains "FileZilla"** – in the program folders, under `AppData`, and directly on every fixed drive. This also finds `C:\FileZilla`, `FileZilla3_x64`, or a portable location.
5	The program's own folder (a portable copy next to it) and the `PATH` variable.
6	As a last resort, a time-limited pass through the drives.

The search happens only once. As soon as FileZilla has been launched, the program remembers the path permanently. On the next launch – and on every further click – it goes straight through step 1, without searching again. If the remembered path is no longer valid one day because FileZilla was moved or reinstalled, the search starts again on its own and remembers the new location.

> **IF FILEZILLA IS NOT FOUND**
>
> The program then offers to download and install it. If you decline, you can select the file `filezilla.exe` yourself – this path is remembered as well.

14.8 ps5_autoloader – the console's startup sequence

After the jailbreak, the PS5 launches a series of payloads – the FTP service, ShadowMount+, kstuff, and whatever else is needed. Which ones and in what order is defined in a simple text file named `autoload.txt`. This window edits it from the PC, over FTP.

Where to find it: MORE TOOLS → ps5_autoloader.

> **THE MOST IMPORTANT THING FIRST: THE SEARCH ORDER**
>
> The console looks for `ps5_autoloader/autoload.txt` in this order: **1. USB stick, 2.** `/data`**, 3. savedata folder** – and uses only the *first* file found. This window works on `/data`. If a stick with its own `autoload.txt` is plugged in, the stick wins, and your changes here have no effect.

How the file is structured

LINE	MEANING
`kstuff.elf`	Launches this file. `.elf`, `.bin` and `.js` are allowed.
`!2000`	Waits 2000 milliseconds before continuing.
`# Text`	Comment, skipped.

The kernel exploit starts on its own and does **not** belong in the list. Anyone using their own ELF loader must name it `elfldr.elf` and place it before the other ELF files.

The buttons

BUTTON	WHAT IT DOES
Fetch from console	Reads the folder and the `autoload.txt`. If a PS5 address is set, this happens on its own when the window is opened.
Write autoload.txt	Writes the edited text back. If it names files that are not actually in the folder, a confirmation is requested first – otherwise the console skips such lines without a word.
Upload file	Places a payload in the folder. Afterward, a check is made whether it carries the execute permission.

BUTTON	WHAT IT DOES
Delete selected	Removes the selected files from the console. With a confirmation prompt.
Save snapshot	Downloads the complete folder into a subfolder `ps5_autoloader` on your PC.
Restore snapshot	Restores a saved state. Files with the same name on the console are overwritten. With a confirmation prompt.

> **WHY THE UPLOAD GOES THROUGH FTPSRV**
>
> A payload only starts if the file on the console carries the execute permission – if it is missing, the console skips the file without any message. The program therefore uploads via `ftpsrv` (port 2121), with which it has been tested, and warns if the permission is missing after the upload.

A good first step: Before changing anything, save a snapshot. Then you can undo any experiment without having to set up the console again.

14.9 AMPR EMU – old and new method

Two buttons that share the same goal and differ in one point that is easy to overlook. Between ShadowMountPlus **1.7 alpha6** and **1.7 alpha8**, the place where the tool looks for the replacement libraries was changed – and a location that was correct before no longer works afterward. No error message: the game simply starts without them.

	OLD METHOD (UP TO ALPHA6)	NEW METHOD (FROM ALPHA8)
Where it looks	`app0/fakelib2`, otherwise `app0/fakelib`	fixed order: `backports/<ID>/fakelib2` → `backports/<ID>/fakelib` → `<Spiel>/fakelib2` → `<Spiel>/fakelib`
`fakelib2` **in the game folder**	takes effect	1.7 alpha8 to alpha13fix1: **is ignored**; from 1.7beta1 (included: 1.7beta12) it takes effect again and overrides a `fakelib` next to it
Folder name in `backports`	does not exist	`<ID>`; from 1.7beta6 also `<ID> Zusatz` (the exact name wins, otherwise the first in byte order)
global + game-specific	two layers on top of each other	previously copied together into a cache
Emulator files	do not exist	from `/data/shadowmount/emus/`

Why always `fakelib` **anyway?** Since 1.7beta1, ShadowMount+ reads a `fakelib2` in the game folder again – and then only that one. The name `fakelib`, however, is read by every version, which is why BACKPORT and AMPR EMU store their files there. If the game already has a `fakelib2`, a note appears in the log before integration. If both folders are present afterward, the warning names the step that is unambiguous in every version: copy missing files from `fakelib2` to `fakelib` and remove `fakelib2`.

Which button? You do not need to know. Both check at startup which version is running on the console and tell you. If you picked the wrong one, a confirmation prompt appears with the reason – and the

advice to use the other one.

Where to find both: behind **button 9** in the CONSOLE view. On the right, it shows the page with both methods and a **Guide** button for each.

Choosing the storage location

ShadowMount+ reads libraries from **three** sources. Up to v1.8.98, the program used only the first; now all three are available on the page. The chosen route can be recognized by its checkmark and is remembered across program restarts.

ROUTE	FOLDER	EFFECT
Per game	The title's backport folder, otherwise `<Spiel>/fakelib`	applies to this one game only
Global	`/data/shadowmount/fakelib`	applies to **every** registered game
Emulators	`/data/shadowmount/emus`	only from alpha8 – and **only replaces what is already there**

> **THE EMULATOR FOLDER DOES NOT ADD ANYTHING NEW**
>
> A file from `/data/shadowmount/emus` is only taken over if it **already exists** in the game's `fakelib`. If you place only `libSceAmpr.sprx` there and the game does not have this library, nothing happens at all – without any message. The folder is intended for *updating*, not for introducing.

If your `config.ini` specifies a different folder (`global_fakelib_path`, `emulators_path`), that one is used – not the default. And if the corresponding switch is turned off, the log says so: otherwise the files could be stored correctly and would still never be used.

What happens when you press it

The button does not open a form; it starts working. Six steps that run along in the window:

STEP	WHAT IT DOES
1	Looks for the PS5: first the address from the settings, then the saved FTP profiles, otherwise the local network. An open port is not enough – a directory must be openable that exists only on a console.
2	Determines which ShadowMount+ version is running there, and names the evidence.
3	Looks for the games – both unpacked folders and titles from mounted images.
4	Determines the storage location. With *Per game*, the version decides this; if you chose *Global* or *Emulators*, your route applies – and the preceding steps are skipped, because no individual game is meant.

STEP	WHAT IT DOES
5	Shows the versions to choose from: `libSceAmpr.sprx` (AMPR EMU) and – only if you check the box – `libScePlayGo.sprx`. In each list, the **newest version is at the top** and is preselected; anyone who changes nothing gets the newest AMPR EMU version without PlayGo. Below the list is what the chosen variant can do (for example "Debug version – writes logs") and where it comes from. The pool is the entire stock: bundled, downloaded, and from your own folders – **excluding the pack builds** (`test-pack`, `test-debug-pack`): they belong to the asset pack at creation time and are not available for selection here.
6	Reconciles the `config.ini` and stores the libraries – afterward the result is read back and verified.

> **QUESTIONS ARE ASKED ONLY WHERE THERE IS SOMETHING TO DECIDE**
>
> And then not with "Yes/No", but with an explanation: under each answer option there is a sentence saying what it means, and the recommended one is marked. Six questions can occur: which console (if several respond), whether the address found should be saved, which game (if several are present), whether to store the files anyway if the game is listed in the `config.ini` under `fakelib_exclude` or is switched off via `fakelib_mode` or `fakelib_default_mode` (from ShadowMount+ 1.7beta4 and 1.7beta5 respectively, ShadowMount+ then mounts no `fakelib` at all for this title – you change the entry under *MORE TOOLS* → **SHADOWMOUNT+**), which versions of AMPR EMU and – if desired – `libScePlayGo.sprx` are stored (a separate window with two lists; Enter accepts the default), and whether deviating keys in the `config.ini` should be adjusted.
>
> Canceling exits immediately at any point, without changing anything.

About the `config.ini`: If a key is not set there, ShadowMount+ uses its built-in default – and that is almost always exactly the right one. A question is therefore only asked if a key is *set* and *differs*. The bundled `config.ini` consists entirely of comment lines, by the way; that is not an error.

Without a console: If the PS5 cannot be reached, the run offers to edit a game folder on this computer instead. You then transfer it to the console yourself. *Global* and *Emulators* do not go this route – those are fixed folders on the console, unreachable without a connection. The run says so and aborts rather than storing anything somewhere.

Both methods remain side by side as long as both ShadowMount+ versions are in circulation.

ShadowMount+ from 1.7beta12 – new format of `config.ini`. Since 1.7beta12 (pre-release of 08.11.2026) `config.ini` has sections (`[general]`, `[emulators]` …); settings per game and per image live in a separate `profiles.ini`. On the first start ShadowMount+ converts an old file itself and keeps a backup in `config-v1-backup/`. The program therefore asks **once** which version is running on the console (before or from 1.7beta12), remembers the answer and writes the file in the matching format – in the ShadowMount+ editor, in the automatic mode of task 7, in the AMPR capture and in the message about AMPR EMU NG (`update_emulators=0` then reads `[emulators] update_enabled=0`). The answer can be changed under *SETTINGS → ShadowMount+ – config.ini*. If the console visibly holds the other format, the program follows the file and says so. Keys that no longer exist in the new format (for example `kstuff_*`) are not written. Checked on the PC with a simulated counterpart, not on the console.

14.10 WebKit Autoloader – Tile on the Home Screen

The WebKit Autoloader creates a tile on the console that repeats the entry point with a single press each time, instead of calling it up by hand in the browser. It consists of two parts: a **host** that runs on the computer and an **installer** that is run once on the console.

Found under: the **CONSOLE** view, button **2. WebKit Autoloader**. The page appears on the right in the view – no separate window; pressing the button a second time changes nothing, *1. Console & payloads* leads back. At the top are the image and the thanks to the author, below them the address of the console (the same as on the other pages – it is needed only for the installer), the choice of version, the three routes and a log with a status line. Showing the page sends nothing over the network.

Under **Version**, choose which version the three routes work with. Several are included – from v0.4.0 to v0.6.1 –, and the newest is preselected, marked *(latest)*. The program remembers the choice. Below it is shown what is available for the selected version: Windows program, Python script and installer, or what is missing. The version is taken from the file names in the folder – so it is exactly the one that is started or sent.

The three routes

If the window is large enough, the three routes sit in a row at the right edge – the installer on the left, next to it the two hosts – and above each button its icon: *With JB ELF* for the installer (console with jailbreak), *Without JB EXE* and *Without JB Python* for the two hosts. The icons are also available with German text (*Mit JB*, *Ohne JB*). In small windows or at low height the icons are omitted, and the buttons wrap so that the log below keeps its room.

ROUTE	WHAT IT DOES
Start host (Windows program)	Starts the bundled host in its own window. It spoofs `manuals.playstation.net` for the console and serves it the autoloader.
Start host (Python script)	The same host as a script – the route for Linux and macOS.
Send installer to the PS5	Gets the `.elf` onto the console. Which route the program takes is decided by the program itself – see below. The large field of the page shows every step: file, version and size, whether port 9021 is open, the sending, “Arrived” with the duration, “Loaded” and the reply of the console (if it reports nothing, the *WebKit Autoload* tile under *Media* shows whether it worked). On the USB route, the field shows the FTP connection, the drive found, the upload and the size check on the console.

> **THE HOST NEEDS ADMINISTRATOR RIGHTS**
>
> It occupies DNS (UDP 53) and HTTPS (TCP 443). Windows gives both ports only to an elevated process. If the start fails, start the program again as administrator. The host stays in its window until it is closed; in the meantime, set the console's DNS to the address it shows.

How the installer gets onto the console

The short route is tried first: if a payload loader is listening on **port 9021** on the console, the file goes directly there.

If this port is silent, the program takes the other route and says so. The file then travels via **FTP** into the **root directory** of a USB drive of the console. **2121** and then **2021** are tried as the port – the first one that answers is used. If several drives are plugged in, the program asks which one it should be.

From there, pick up the file on the console with the **Payload Manager** and send or install it.

> **AFTERWARDS**
>
> The **WebKit Autoload** tile is on the home screen of the console under **Media**.

Adding a newer version

The installer of every version is in the folder `PS5 WebKit Autoloader` next to the program: `webkit-autoloader-installer_vX.Y.Z.elf`. The two hosts (`webkit-autoloader-host_vX.Y.Z.exe` and `.py`) are not bundled with the program: on the first click on **Start host**, it downloads the host of the selected version from the author's release (with a progress display, the SHA-256 is verified) into the settings folder and starts it right away; every further click starts the saved file. If the file is gone afterwards, the antivirus has usually removed it – allow it there if you trust it. When a new version appears, put its installer **in addition** into that folder; the older ones stay and remain selectable. The program reads the folder when the page is built: the new version then appears in the selection, the newest first (sorted by the numbers in the name: *0.10.0* comes after *0.9.0*). Nothing has to be changed for this. A file without a number in its name does not appear in the selection; it is used only if there is no version with a number at all. At the next build, the folder contents go unchanged into the finished program.

14.11 Send unjail – Lifting Homebrew out of the Sandbox

Homebrew applications run on the console in a folder of their own and may write only there. Some need more – they then ask for extended access. **unjail** is a small console daemon that answers exactly these requests. It is sent once and then runs until the next restart of the console.

Found under: MORE TOOLS → Send unjail. Enter the address of the console and click **Send to PS5** – progress and the reply of the console appear in the window meanwhile.

It makes sense *before* starting a homebrew that needs extended write access. For ordinary conversions with this program, unjail is not needed.

> **ONLY UP TO FIRMWARE 10.60**
>
> unjail is verified for firmware **1.00 to 10.60**. From **11.00** on, the daemon aborts at startup before it is ready – on a console with 11.00 or newer it currently does not work. The window points this out before sending.

14.12 PS5 Wee Tools – the Console's NOR Flash

PS5 Wee Tools by andy-man is a bundled third-party tool (GPL-3.0) – not part of the conversion. It does not work with game dumps but with the **NOR flash of the console**: from a NOR dump it reads model, SKU, region, serial numbers, firmware and MAC addresses, splits the dump into its partitions and reassembles it, shows the UART log including EMC error codes, and can read and write the NOR via an SPI flasher (SPIway on a Teensy 2.0).

Found under: MORE TOOLS → PS5 Wee Tools (NOR). After a confirmation prompt, the tool opens in a console window of its own, on Linux and macOS in a terminal. The start can take up to a minute: on Windows and Linux, the program file unpacks itself a second time for this (measured on Windows: 19 to 48 seconds). Do not click again in the meantime – a second start opens a second window. It is operated there with numbers and letters; **6** in the main menu quits it.

It stores read NOR dumps (`dump_*.bin`) and UART logs (`uart_*.txt`) in the **PS5 Wee Tools** folder next to the program (on macOS in the settings folder); it also remembers language, sound and connections there. On the first start, it takes over the language from the program.

> **A NOR WRITTEN INCORRECTLY MAKES THE CONSOLE UNUSABLE**
>
> Reading is harmless. *Writing* the NOR and switching flags (kiosk mode, model, active slot) change the console's firmware memory. Keep a backup of the original dump beforehand. The flag switch writes without confirmation directly into the opened file – so work with a copy.

14.13 ELF → EBOOT.BIN – Packaging an ELF as eboot.bin

Packages an executable ELF file – for example a self-built homebrew – as `eboot.bin` : a SELF wrapper with the ELF inside, the way the PS5 launches it.

Found under: MORE TOOLS → ELF → EBOOT.BIN. Choose the `.elf` , then the save location (the suggestion is `eboot.bin` in the same folder). Packaging uses the same method with which BACKPORT (14.6) resets files. Afterwards the program reads the result back as a check; only when it appears there as a SELF with an embedded ELF does the window report success – with the size before and after and the number of segments.

- **For jailbroken consoles only.** The `eboot.bin` is a fake SELF without a real signature and is not compressed. The packaging itself is tested (BACKPORT has used it for a long time); the program has not yet tested launching an `eboot.bin` created this way on the console.
- **What is rejected:** a file that is already a SELF (it does not need to be converted), a stripped ELF without an identifier at the start of the file, and anything that is not an ELF.
- **No half-finished state.** Writing goes to a temporary file first, and only then does it replace the target. Even if you choose the source itself as the target, it stays intact until the very end.

The structure of the result is shown by the SELF inspector (14.1).

14.14 EDIT IMAGE – Changing Files in an Image

Rename, delete, replace or add files of a finished image – in two ways: **Edit in place** writes into the image itself (`.exfat` , `.ffpkg` and uncompressed `.ffpfs`), **Unpack** takes the route the program takes elsewhere too: unpack, change, repack – for every format, `.ffpfsc` included.

Found under: MORE TOOLS → EDIT IMAGE.

Edit in place (without unpacking and rebuilding):

1. **Choose the image, *Edit in place*.** The program asks whether the image itself really should be changed and then shows its content in the tree – read from the image, without unpacking.
2. **Change** as below (*Rename*, *Delete*, *Replace …*, *Add …*, *New folder*). Every change is only queued at first: the tree already shows the new state, while the file stays untouched. Whatever is not possible (name already taken, no room) is rejected – and then nothing is queued either.

3. **Apply changes.** Only now is anything written, in the background: first the old bytes of every affected spot into a journal next to the image (`<Abbild>.undo`), then the new content, and finally the file system's management data. If writing is interrupted (crash, power loss), the program reverses it the next time the image is opened, using the journal, and tells you so. A change that has been applied completely cannot be taken back this way. If you close the window with queued changes, the program asks and discards them.

- `.exfat` never grows. Images made by this program have no free cluster; room is made by deleting or by replacing with a smaller file.
- `.ffpkg` (UFS2) never grows either, but has some reserve because the program adds room when building it. It recalculates the bitmaps and counters of the file system after every change the same way it creates them when building.
- `.ffpfs` (flat PFS – this is how the program builds `.ffpfs`): new content goes into free gaps or is appended at the end, so the image grows as needed; the space of a deleted or replaced file is used again next time. The program rebuilds the directories and the path table when applying, exactly as MkPFS would build them for the same content. A PFS knows no empty folders: a new folder needs at least one file by the time you apply. An `.ffpfs` that merely wraps an inner exFAT image is changed like an `.exfat`.
- **Not in place:** `.ffpfsc` (compressed) cannot be edited this way – use *Unpack* for it.
- **Tested** is the in-place route with independent readers: exFAT with the exFAT driver of the Linux kernel, UFS2 with UFS2Tool (`fsck_ufs`, `extract` and a probe that fills the free space and then compares all files), PFS with the MkPFS check against a folder with the same changes. An `.ffpkg` or `.ffpfs` changed this way has not yet been measured on the console – try with a copy first.

Unpack (every format):

1. **Choose and unpack the image.** The program unpacks it with Task 6 (target dump folder) into a work folder – with all the checks and prompts of that task; progress and log appear as usual in the main window.
2. **Change.** Select an entry in the tree: *Rename*, *Delete*, *Replace* … (the content of the file is replaced by another one, name and location stay), *Add* … (new files into the selected folder) and *New folder*. Every change takes effect immediately in the work folder.
3. **Save.** Choose the format (the original's is preselected), *Save as image* …, choose the target folder. The result is named `<Name>_bearbeitet.<Endung>`; **the original stays unchanged.** It is also possible to save to a different format or to save several times.

- **Protected** are `eboot.bin` and `sce_sys/param.json` – on both routes: they can be replaced (for example by a patched `eboot.bin`) but not deleted or renamed, and neither can the folder that contains them. If one of them is missing when saving, the program asks first.
- **Names** must be valid on the console and under Windows; two names that differ only in upper/lower case count as the same. Links are not touched, and nothing can leave the game folder.
- **Space:** The work folder (unpacked game) and the new image need space together; the work folder is in the configured temp folder and is removed when the window is closed. If there are changes not yet saved, the program asks first.
- **Tested** is the whole route on a small `.ffpfsc` (unpack, replace, rename, add, save, unpack the result again and compare). Whether a game changed this way starts on the console depends on what you changed – try with a copy first.

14.15 Devices on the network – NAS and Computers as Source or Target

Finds NAS devices and computers on your own network and sets their Windows share as the source or target. **Found under:** MORE TOOLS → Devices on the network.

1. **Search.** The search takes a few seconds; it finds whatever announces itself on the network (mDNS and WS-Discovery). The network is not scanned, and no addresses are tried out. *Stop* ends the search.
2. **Choose as source … / Choose as target …** opens the folder dialog on the device's share (`\\Adresse\`); the chosen folder goes into the SOURCE or TARGET FOLDER field. The program reads and writes there without an intermediate copy.
3. **Copy address** puts the address on the clipboard, for example for another program.

- Devices with a Windows share (SMB) and computers can be chosen. FTP and NFS devices are only displayed.
- If the search finds nothing, the device's network discovery is usually switched off. A share can also be entered into the field by hand as `\\Adresse\Freigabe`.
- Tested is reading and writing through a share on your own computer (`\\localhost`); it has not yet been measured with a real NAS.

15 Theme and Background Image

The **THEME** button offers seven color schemes:

THEME	CHARACTER
Futuristic (Neon Cyan)	Glowing neon cyan on teal black.
Dark	Near black, glossy, blue accent – default setting.
Light	Warm, creamy white with dark text.
Metallic	Brushed charcoal metal with silver and steel blue.
Calm Graphite	Calm charcoal with a steel-blue accent. The menu bar at the top has just one text color (instead of FILEZILLA green, JS LOADER orange, CREDITS blue), the page heading has the same color on all pages, and the secondary buttons are slate gray instead of turquoise. Layout and operation are the same in all themes.
Midnight Blue	Deep night blue with a light blue accent; menu bar, heading and secondary buttons as with Calm Graphite. A dark background image goes well with it, for example `bg_01_navy-curves`.
Light and Clear	Cool white with dark text and a blue accent; menu bar and heading uniform. A light background image goes well with it, for example `bg_07_porcelain` with `sidebar_07_porcelain`.

The choice is saved per user. After a click on **APPLY**, the program restarts automatically so that the interface appears fully in the new theme. If a task is running, the restart is postponed – please restart by hand afterwards.

Buttons, input fields and drop-down lists are rounded (pill shape) in all themes – in both views, CONVERT and CONSOLE. The sidebar is deliberately designed differently: the switch between CONVERT and CONSOLE is a tall, rounded surface with the small line “SWITCH AREA” above the name of the other area. The tasks, the tools of the CONSOLE view and the two buttons at the bottom edge appear as a list with fine lines and left-aligned text; the selected row carries a bar in the accent color, and the row under the mouse pointer is slightly tinted. The sidebar image is dark in all themes; if the text of a theme would not be legible on it (for example in the light theme), it automatically becomes light, and the switch carries the colors of the dark theme there. The rows themselves stay transparent.

Headings, labels and hint texts appear directly on the background image, with no surface behind them. The dial for the workers also sits freely on the image.

Title bars. On Windows 11, the title bars of all windows of the program are dark gray-blue, with light text and light window buttons; in the themes **Light** and **Light and Clear** the bar stays light. When the theme is changed, open windows change color immediately. On Windows 10, Linux and macOS this option does not exist; the system's bar remains there. The standard Windows dialogs (choosing a file or folder) belong to the system and stay light.

> **LARGE SCREENS AND HIGH DISPLAY SCALING**
>
> The interface follows the display scaling that is set in Windows. Text and controls grow together: up to **125 %** you see exactly the familiar picture, above that buttons, corners and windows become correspondingly larger. On a 4K screen at 200 %, a 44 pixel tall button becomes one of 70 pixels.
>
> If you want to determine the scaling yourself, set it under **SETTINGS** (section 14.3); automatic remains the default.
>
> Should a label ever hit its edge, the **DIAGNOSTICS** button reports this in the section “Display check” – with the name of the element and the number of pixels missing.

14.1 Choosing Background Images

Under **SETTINGS** there are two separate areas, each with its own drop-down list of the bundled images:

AREA	IMAGES	BUNDLED
Background image	the wide images for the main area (20 images)	landscape, 1427 × 1111 pixels
Sidebar background image	the tall images for the sidebar (20 images)	portrait, 493 × 1111 pixels

How large an image *should be* depends on your screen and is therefore shown directly in the settings window under each of the two areas. The main area lies to the right of the sidebar and therefore needs the width of the maximized window minus the sidebar; the sidebar needs its actual width, and both are as tall as the window. On a screen of 1920 × 1200 pixels at 125 % display scaling, the maximized window is 1920 × 1111 pixels and the sidebar is 493 pixels wide: the main image is **1427 × 1111**, the sidebar image **493 × 1111** pixels (this is how the bundled images are made). Both images lie separately side by side, neither under the other. An image of exactly these dimensions is neither scaled up nor cropped; if you bring one to these dimensions in a graphics program, it is best to stretch it without keeping the aspect ratio. A smaller image is accepted but then looks softly stretched; a larger one is cropped from the center.

The file names carry a number and a name, for example `bg_01_navy-curves` for the main area and `sidebar_01_navy-curves` for the sidebar. **The same number and the same name** means: the two

belong together and are color-matched to each other – this exists fifteen times (01 to 15). Five main images and five sidebars (16 to 20) have no counterpart. The matching is not required anywhere; every combination is possible.

Select an image, click **Apply selected** – done, no restart. With **Choose image…**, a custom image can be used instead; **Reset** restores the initial state. The **Save** button at the bottom saves the state and closes the window; it is not required, because every change already takes effect on click.

The images appear as they are in the file; brightness and contrast can be adjusted under **Appearance** (section 14.2). The labels sit directly on the image – so how bright it is matters:

LIGHT IMAGES BELONG TO THE LIGHT THEME

The themes Futuristic, Dark, Metallic, Calm Graphite and Midnight Blue write in *light* text. On a light image this text disappears: `bg_07_porcelain`, `bg_12_ice-light` and `bg_19_frost-waves` as well as `sidebar_07_porcelain`, `sidebar_12_ice-light` and `sidebar_16_sand-dunes` are therefore intended for the themes **Light** and **Light and Clear**, which write in dark text. Borderline are `bg_04_mist-blue`, `bg_14_sunset-silk` and `sidebar_04_mist-blue`.

The settings window points this out under the preview as soon as a light image is selected; the diagnostic report also reports it (*Image brightness*). Anyone who still wants a light image in a dark theme lowers the image brightness under **Appearance** – the note then disappears on its own.

WHY TWO SEPARATE LISTS

The two areas have very different formats. A tall sidebar image in the wide main area would have to be cropped so heavily that hardly any of the motif would be left – it is never distorted, but it does not make sense either. That is why each list shows only what fits there. The distinction is made by the format of the image, not by the file name; a custom image therefore automatically lands in the right place.

METADATA FROM THE INTERNET

If a backup is missing the **title**, **publisher** or **category**, the program can look them up. Whether it does so on its own has depended on the system since v1.8.92: on **Windows and Linux** it looks them up on its own, on the **Mac** it does not. Where it does not happen on its own, the *Game info* window shows a button **Look up missing details online**; only a click on it opens a connection. If nothing is missing, the button does not appear at all.

In doing so, the **title ID** of the game goes out – that is, the information about which game you are working on. The queries go to store.playstation.com and to prosperopatches.com (PS5 titles) or orbispatches.com (PS4 titles). Details fetched once are then kept on your disk for 30 days and cost no second connection.

This can be changed in the **Settings** in the section **METADATA FROM THE INTERNET**. By default the box is checked on Windows and Linux and empty on the Mac. As soon as you set or clear it yourself once, **your** choice applies – on every system and across program starts.

With a firewall (such as Little Snitch on the Mac): the prompt appears only when you press the button. If you allow the connection there permanently once instead of just once, it stays quiet afterwards.

14.2 See-through, Brightness and Contrast

In the settings window, a row of sliders is under **Appearance**. They determine how strongly the background image shows through the individual surfaces and how the image itself looks. A value is applied when the slider is *released* – while dragging, every intermediate image would be recalculated, and that stutters.

SLIDER	WHAT IT DOES	DEFAULT
Path card	How strongly the image shows through the card with SOURCE, TARGET FORMAT and the other fields. 0 % lets the surface stand on its own, 100 % shows only the image.	100 %
Bottom button bar	The same for the surface of the bar that holds progress, percent and size display. It starts only 8 pixels to the right of CANCEL: behind START and CANCEL there is always the image itself, even at a low value.	100 %
Status log	Pulls the color of the log field toward the color of the image.	0 %
Background image: brightness / contrast	Affects the image in the main area – and thus also the sections that the card and button bar show from it.	100 %
Sidebar: brightness / contrast	The same for the sidebar image, independent of the main area.	100 %

> **WHY THE STATUS LOG REACTS MORE WEAKLY**
>
> It is a text field and thus opaque – an image cannot truly show through there. The slider instead pulls its surface color toward the color of the image. With a very dark image, the difference is therefore small; that is not an error.

With the default values, the interface appears unchanged: card and button bar show the image completely, the status log keeps its theme color, and both images stay as they are in the file. The sliders therefore take effect only when you move them. The button **Reset all sliders** sets all seven back to this default.

Next to the two drop-down lists there is also a **thumbnail preview** of the currently selected image, so that a file name is not the only thing to go by. It shows the image already with brightness and contrast, as it arrives in the window.

14.3 Setting Scaling and Resolution by Hand

The program adapts to the screen on its own: it takes the scaling from the system, and the window sizes follow the detected screen size. Anyone who wants to intervene by hand – for example because a different screen is connected or the interface looks too small or too large – finds the section **Display: scaling and resolution** at the very top under **SETTINGS**.

SETTING	WHAT IT DOES	OPERATION
Interface scaling	Size of text and controls, in percent as in the Windows display setting (100 % is an unscaled screen). Below 125 % only the text shrinks; above it, buttons, corners and windows grow too.	Slider in steps of 5 %, from 100 % up to at most 300 %.
Screen resolution	The screen size that windows and background images are sized for: the recommendation for the image size follows it, and no window becomes larger than this screen. If it is smaller than the real one, the main window opens at the size of a window maximized there, centered, instead of maximized. It does not change text or buttons.	Drop-down list; the first entry is *Automatic* with the detected resolution.

Automatic remains the default. As long as nothing is moved, the slider and the list show what the system supplies (for example 125 % and 1920 × 1200). Only what is chosen by hand applies – and at every start, even if the screen changes, until **Reset to automatic** is clicked.

Saving is done with **Apply and restart**. The scaling takes effect only at startup, because fonts are converted to pixels when they are created; a half-hearted adjustment while running would be worse than none. If an operation is currently running (a PKG merge, a transfer) or a tool window cannot be closed, the setting is only saved: it applies from the next start. The sentence under the sliders states what is saved; the note *Not applied yet* is shown as long as the running session does not have it yet. **Reset to automatic** deletes both values immediately and offers the restart if this session ran with hand-set values.

> **WHY THE SLIDER DOES NOT GO UP TO 300 %**
>
> The interface becomes wider with the scaling, in the same proportion (measured: 966 pixels at 125 %, 1532 at 200 %, 2292 at 300 %). If it extends past the side of the screen, there is no scroll bar to the right, and the button that undoes the choice could no longer be reached. The slider therefore goes only as far as the interface fits into the width of the screen: up to 190 % at a width of 1920 pixels, up to 135 % at 1366 pixels. The value that the system itself supplies always remains selectable. If a smaller resolution is chosen, the limit drops with it.
>
> In height it is enough for the column to scroll: if the window is too low, a scroll bar appears on the right.

The diagnostic report names the setting (*Scaling (setting)* and *Resolution (setting)*) and notes that it is set by hand. If the chosen scaling does not take effect, it warns.

16 Solving Common Problems Easily

PROBLEM	WHAT YOU CAN DO
Antivirus reports or removes the EXE	Programs built with PyInstaller are occasionally flagged by mistake. The program does not add any exclusions to your antivirus software on its own and does not install any certificates. Download the EXE only from the project's release page; if the source is right, you can add the EXE as an exclusion in your antivirus software yourself.

PROBLEM	WHAT YOU CAN DO
START stays disabled	Check that the source and the target folder have both been chosen completely; the temp folder may stay empty. Task 8 does not need a target folder.
Source is rejected	Check whether the selected task matches the file type. If necessary, use Task 6 or check the source with Task 8.
“sce_sys/param.json is missing” with a Yes/No prompt	The game is incomplete. If you choose **Yes**, the program creates a valid `param.json` and the build continues; the title ID comes from `sce_sys/nptitle.dat`, or from the folder name as a fallback. If *Look up missing metadata online* is enabled in the settings (by default on Windows and Linux), the title and content ID are looked up at the same time. If you choose **No**, the build is aborted – it is best to make a new backup of the original then.
Content ID in the `param.json` (“param.json has no Content ID”)	The title and content ID are stored in **no** file of a backup – not even in the `eboot.bin`. Only the title ID can be found locally. With the *Look up missing metadata online* checkbox enabled in the settings, the program looks up both; only the title ID is sent to prosperopatches.com in the process. Without the checkbox, without internet, or if the site does not respond, the file receives a **placeholder** as the content ID (`UP0000-<Title-ID>_00-…`) – without a content ID, tools that build a PKG from it abort. If it is missing in an existing `param.json`, the program offers to enter the placeholder; the old file is kept as `param.json.alt`. For updates and DLC, the real content ID must be entered.
Not enough disk space	Free up space or choose a different target or temp folder. Only then repeat the operation.
The operation takes a long time	Watch the progress bar, the status line, and the log. Large files need longer write and verification phases.
The operation was canceled	Wait for the cancellation confirmation. Delete incomplete results and start again with verified paths.
A warning appears	Read the full message in the log. Do not change several things at once; first fix the cause that is named.
Game does not start (`CE-108255-1`)	Two known causes: an incomplete `.ffpkg` – or starting from internal storage instead of from the USB stick. Try the backup from the stick.
Update package cannot be installed (`CE-107891-6`)	An update package (delta) contains only the changes and belongs to exactly *one* base package. A different one is installed – even a rebuild of the same game counts as a different package. Install the base package the update was built against, or rebuild the base package and the update together and install both. Whether a `.pkg` is an update package is shown by **PKG** (Task 7) in the *Parts* column (number of updates per game).
FTP, KLog, or JS loader tool does not connect	Check the IP address and port, and whether the PS5 has been prepared by jailbreak and is reachable on the same network. An FTP payload must be running on the console – one can be started via **JS LOADER**.

16.1 What belongs in a helpful error report?

If you need support, note the program version (v1.9.97), the selected task, the source type, and the last visible log message – the easiest way is the **DIAGNOSTICS** button. Do not pass on private paths or personal data if they are not needed for troubleshooting.

PLEASE DO NOT DELETE RIGHT AWAY

Keep the log and the unchanged source until you have understood the cause. This makes an error easier to trace.

16.2 The environment check (“Doctor”)

If something does not work although it works for others, the environment check answers the most common causes in a few seconds. It is called from the command prompt or from PowerShell, in the program’s folder:

```
PS5_Dump_Image_Converter.exe --doktor "E:\PS5_Temp" "E:\Ziel"
```

Both folder arguments are optional. Without them, the system’s temp folder is checked.

OPEN AS ADMINISTRATOR

The program always requires administrator rights – this applies to every call, including the environment check. Therefore open the command prompt or PowerShell yourself as administrator, otherwise Windows refuses to start it.

Each line starts with its rating: `[ ok ]` fine, `[ !! ]` a note, `[FEHL]` a real error, `[ -- ]` not needed or cannot be determined. Among other things, the following are checked:

- **Long paths.** If they are turned off in Windows, packages with deep folder structures abort – often with a message that sounds like something completely different.
- **The file system of the folders.** On FAT32, every file ends at 4 GB; a dump is always larger, and the abort would only come after hours.
- **Free space and write permission** for the temp and target folders.
- **The bundled programs** – whether they can be started at all.
- **The settings file.** If it is damaged, the program starts with default values; it then looks as if it had forgotten everything.

The output is suitable for copying into an error report. It contains no credentials and makes no connection to the internet. The same information is shown in the **DIAGNOSTICS** window in the *Doctor* section.

17 Terms Explained Briefly

The following terms appear regularly in the interface. The explanations are deliberately kept short.

TERM	EXPLANATION
Dump folder	A normal folder containing the files and subfolders of a dump.

TERM	EXPLANATION
`.exFAT`	An image file in exFAT format.
`.ffpfsc`	A compact, compressed container file in which a dump is bundled.
`.ffpfs`	Like `.ffpfsc`, but uncompressed – therefore larger, but faster to create.
`.ffpkg`	A UFS2-based container file. The result is verified automatically after writing.
Compression level	Determines how strongly the `.ffpfsc` container is compressed (1 = fastest, 9 = smallest file). With `.exFAT`, `.ffpkg`, `.ffpfs` or a dump folder as the target the field is locked, as nothing is compressed there.
Worker threads	Number of copy and compression operations working at the same time. Higher values can be faster but need more system resources.
Source	The folder or file that you want to process or check.
Temp folder	A work folder for temporarily needed files. It should have enough free space.
Target format	The format that is to be produced after a conversion.
Target folder	The folder in which the finished result is saved.
FTP	Protocol for exchanging files with a PS5 reachable over the network – used by FILEZILLA, SHADOWMOUNT+, and the library comparison.
KLog	The PS5's kernel log, a technical live log of the console – useful for troubleshooting homebrew applications.
SDK	The development package with which a game was built. The console compares this value with its firmware at startup – see BACKPORT.

18 Quick Reference for Printing

NO.	TASK	USE IT FOR
1	Convert dump folder	Folder to `.ffpfsc`, `.ffpfs`, `.exFAT`, or `.ffpkg`.
2	Convert ffpfsc	Unpack or convert `.ffpfsc`.
3	Convert exFAT	Unpack or convert `.exFAT`.
4	Convert ffpkg	Check, unpack, or convert `.ffpkg`.
5	Batch conversion	Process several dump folders and containers in one batch.
6	AIO conversion	Automatically detect and export a supported source.

NO.	TASK	USE IT FOR
7	PKG / FPKG	No function yet.
8	Validator	Check a source without creating a conversion result.
–	AMPR EMU Manager (CONSOLE view)	Choose, apply, or reset the AMPR/PlayGo version; create the index – also directly on the PS5.

A RULE OF THUMB FOR GETTING STARTED

Keep a copy → choose the matching task → use a separate target folder → plan for enough disk space → wait for the success message.

19 The Application on Linux

The program runs not only on Windows but also on Linux. The interface, all eight tasks, and the additional tools look and work there exactly as described in this manual. This chapter covers only the differences.

19.1 Building the program file

Unlike on Windows, there is no ready-made file to download: a Linux program is tied to the architecture and the system libraries of the computer on which it was built. It is therefore built on your own system. This takes a few minutes and is done with two commands in the project folder:

```
chmod +x Build_Linux.sh
./Build_Linux.sh
```

The script checks the prerequisites, fetches the required packages, and places the finished file under `dist/`. If Tcl/Tk – the foundation of the interface – is missing, it gives the appropriate install command for your system.

DO NOT BUILD WITH SUDO

Building does not need elevated rights. If it is started as `root` anyway, the result folder afterward belongs to `root` and can no longer be cleaned up on the next run.

19.2 Starting and adding to the menu

The built file can be started directly. An entry in the application menu is more convenient:

```
chmod +x Install_Linux.sh
./Install_Linux.sh
```

Afterward, "PS5 Dump & Image Converter" appears in the menu with its own icon and can be launched in the terminal with `ps5-dump-image-converter`. Everything is placed under `~/.local` – without `sudo` and without touching the system. You can undo this with `./Install_Linux.sh --entfernen`.

19.3 What is not available on Linux

Two paths depend on programs that exist only for Windows. If you choose such a task, the program says so explicitly in the log.

AFFECTED	REASON
Reading and creating `.ffpkg` – that is, **Task 4** and every `.ffpkg` as a target format	These paths run through UFS2Tool and the Dokan driver. Both exist only for Windows.
The fallback paths via OSFMount	OSFMount exists only for Windows. These paths only come into play when the normal path fails on an unusual image.

All other tasks – dump folder, `.ffpfsc`, `.ffpfs`, and `.exFAT` in every direction, batch conversion, the AMPR EMU Manager, and the Validator – are fully available.

19.4 Other everyday differences

AREA	ON LINUX
Elevated rights	Not needed. On Windows, the program asks for them at startup; on Linux it starts as a normal user.
Temp folder	Preset to `/tmp`. Do not put your **target folder** there – the area is cleaned up after the work is done.
Settings	Located under `~/.config/PS5ImageConverterPro/`.
Typeface	The interface is designed for *Segoe UI*. If this font is not available, the program itself picks the best match on your system.
Opening the manual and folders	Works as usual through the default programs of your desktop environment.
Shutting down the computer	The checkbox from chapter 3.2 works here as well.
FileZilla	Is found if it is installed through the package manager. Automatic installation from within the program exists only on Windows.

SHARING

A program file built on Linux does not necessarily run on another computer – older systems ship with older system libraries. When in doubt, rebuild it on the machine where it is supposed to run.

20 The Application on the Mac

The program also runs on a Mac with macOS 11 (Big Sur) or newer – on Apple silicon as well as on Intel models. The interface, all eight tasks, and the additional tools look and work there exactly as described in this manual. This chapter covers only the differences.

20.1 What you need beforehand

A Mac does come with a Python, but it is unusable for building the program: it lacks the foundation for the windows. Therefore install a complete Python beforehand – either the installer from `python.org` or via Homebrew with `brew install python-tk`. The build script checks this and tells you exactly what is missing before it begins.

20.2 Building the program

As on Linux, there is no ready-made file to download: the program is tied to the architecture of your Mac. A version built on Apple silicon does not run on an Intel Mac and vice versa. Two commands in the project folder are enough:

```
chmod +x Build_macOS.sh
./Build_macOS.sh
```

This takes a few minutes. At the end, the finished program `PS5 Dump & Image Converter.app` is under `dist/` – an application bundle, as you know it from any Mac program. A double-click starts it.

DO NOT BUILD WITH SUDO

Building does not need elevated rights. If it is started as `root` anyway, the result folder afterward belongs to `root` and can no longer be cleaned up on the next run.

20.3 Placing it in the Applications folder

So that the program sits in its usual place in Launchpad and in the Finder:

```
chmod +x Install_macOS.sh
./Install_macOS.sh
```

If the `Programme` (Applications) folder is locked, the script places the program in your personal Applications folder. This needs no password and appears in Launchpad just the same. You can undo this with `./Install_macOS.sh --entfernen`; your settings are kept.

> **"THE DEVELOPER CANNOT BE VERIFIED"**
>
> This message appears when the program reached you via browser, mail, or AirDrop – macOS flags everything that comes from outside. The install script removes the flag itself. By hand, it works like this:

```
xattr -dr com.apple.quarantine "/Applications/PS5 Dump & Image Converter.ap
```

20.4 What is not available on the Mac

These are the same two paths as on Linux, and for the same reason. If you choose such a task, the program says so explicitly in the log.

AFFECTED	REASON
Reading and creating `.ffpkg` – that is, **Task 4** and every `.ffpkg` as a target format	These paths run through UFS2Tool and the Dokan driver. Both exist only for Windows.
The fallback paths via OSFMount	OSFMount exists only for Windows. These paths only come into play when the normal path fails on an unusual image.

All other tasks – dump folder, `.ffpfsc`, `.ffpfs`, and `.exFAT` in every direction, batch conversion, the AMPR EMU Manager, and the Validator – are fully available.

20.5 Other everyday differences

AREA	ON THE MAC
Elevated rights	Not needed. On Windows, the program asks for them at startup; on the Mac it starts as a normal user.
Temp folder	Do not put your **target folder** in a temp area – it is cleaned up after the work is done.
Settings	Located under `~/Library/Application Support/PS5ImageConverterPro/`.
Typeface	The interface is designed for *Segoe UI*. If this font is not available, the program itself picks the best match on your system – on a Mac usually *Helvetica Neue*.
Opening the manual and folders	Works as usual. "Show in folder" opens the Finder and selects the file in it.

AREA	ON THE MAC
Shutting down the computer	The checkbox from chapter 3.2 works here as well. The first time, macOS asks whether the program may control other programs – confirm this, otherwise the computer stays on.
FileZilla	Is found if it is installed through Homebrew. Automatic installation from within the program exists only on Windows.

SHARING

`./Build_macOS.sh --dmg` additionally creates an image (`.dmg`), as you know it from downloading other Mac programs. Keep in mind: it only fits Macs of the same architecture as yours.

This manual describes the visible operation of the program. Additional version and feature information is in the README.md and CHANGELOG.md shipped together with it.